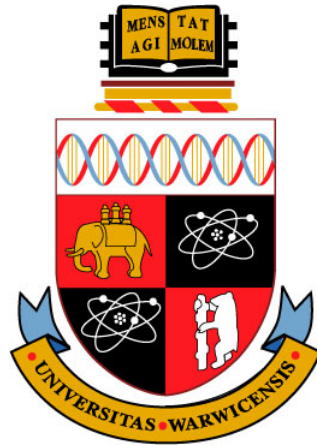




Budgeted Influence Maximisation in Social Networks

CS913 Dissertation Project

Calum Harvey

Supervisor: Dr. James Archbald

Department of Computer Science

University of Warwick

September 2021

Abstract

The budgeted influence maximisation problem is a generalised version of the influence maximisation problem which consists of selecting a small subset of nodes within a network that maximise the spread of influence without exceeding a budget, given each node has an arbitrary cost. Finding the subset of nodes, called the seed set, was found to be NP-Hard which has prompted exploration into techniques for approximating the optimal seed set. Within this project, a number of novel heuristic approaches are proposed in addition to a different method of node costing. Three heuristics, named Lowest Cost, Highest Cost, and Average Cost, are detailed that solely utilise a nodes cost for seed selection, with two further heuristics PICR and PICRnp proposed. The performance of these heuristics is then compared against previously developed heuristics for influence maximisation.

The PICR and PICRnp heuristics determine the ratio between the potential influence a node possesses and the cost of the node, allowing the cost-effectiveness of a node to be determined, with the most cost-effective nodes added as seeds. Experiments undertaken utilise three non-synthetic large-scale social networks and the Linear Threshold, Independent Cascade, and Weighted Cascade diffusion models. Additionally, three cost functions are used to determine the cost of each node in a network: random, degree, and a newly proposed method of costing using PageRank.

Results showed that Lowest Cost, PICR, and PICRnp heuristics obtain the best performance over all experiment environments, outperforming all previously developed heuristics implemented in this project. Additionally, Lowest Cost had the greatest consistency throughout all experiments conducted. Moreover, the use of PageRank allowed for heuristics to be tested for greater generality as different cost functions allow for a more diverse range of scenarios.

Keywords: Budgeted influence maximisation, social networks, information diffusion

Acknowledgements

First of all, I would like to thank my supervisor Dr James Archbold for overseeing the project. I would like to express my gratitude for the time he dedicated to providing feedback such that the project could be completed successfully.

Second, I would like to thank my family and friends, who supported me during my Masters study.

Abbreviations

IM	Influence Maximisation
BIM	Budgeted Influence Maximisation
LT	Linear Threshold
IC	Independent Cascade
WC	Weighted Cascade

Contents

Abstract	ii
Acknowledgements	iii
Abbreviations	iv
List of Figures	vii
List of Tables	vii
List of Algorithms	vii
1 Introduction	1
2 Literature Review	3
2.1 Networks	3
2.2 Influence Maximisation	4
2.3 Influence Models	4
2.3.1 Linear Threshold (LT)	5
2.3.2 Independent Cascade (IC)	5
2.3.3 Weighted Cascade (WC)	5
2.4 Uniform Cost	5
2.4.1 Greedy	6
2.4.2 Heuristics	6
2.5 Non-Uniform	7
2.6 Costing	8
3 Project Management	10
3.1 Methodology	10
3.2 Timeline	10
3.3 Tools	12
3.3.1 NetworkX	12
3.3.2 Network Diffusion Library	13
3.3.3 Python Libraries	13
3.3.4 SNAP	13
3.3.5 Git	13
3.4 Risk Management	13
3.4.1 Encountered Risks	14
4 Design	15
4.1 Networks	15
4.2 Models	15
4.3 Heuristics	16
4.3.1 Highest Cost	17
4.3.2 Lowest Cost	18
4.3.3 Average Cost	18
4.3.4 Potential Influence Cost Ratio	19
4.4 Cost Functions	21
4.4.1 PageRank	21

5	Implementation	24
5.1	Networks	24
5.2	Models	24
5.3	Heuristics	25
5.4	Cost Functions	26
5.5	Testing	26
6	Experiments	28
6.1	Parameters	28
6.2	Experiment Setup	29
7	Results	30
7.1	Max Influence Spread	30
7.2	Lowest Cost Performance	32
7.3	Highest and Average Cost Performance	34
7.4	PICR and PICR without probability	34
7.5	Differences Between Degree and PageRank	35
8	Conclusion	38
8.1	Findings	38
8.2	Future Work	38
9	Evaluation	40
9.1	Project Evaluation	40
9.2	Research Evaluation	40
	Appendices	44
A	Full Results	44

List of Figures

3.1	GANTT chart of project timeline showing time and duration of each phase.	12
3.2	Updated GANTT chart of project timeline with testing moved to phase two.	12
4.1	Example of PageRank Algorithm	22
7.1	Results for the GitHub Network using random costing on the Linear Threshold model.	30
7.2	Graph showing power-law with node degrees on the x-axis and the fraction of nodes at each degree on the y-axis. The green line displays a power law relationship and the blue line shows the degree distribution of the GitHub network.	31
7.3	Results for the Amazon Network using degree costing on the Weighted Cascade model	32
7.4	Chain of directed nodes	33
7.5	Results for the GitHub Network using degree costing on the Linear Threshold model	34
7.6	Different scenarios for single degree nodes	36

List of Tables

4.1	Network node and edge counts	15
4.2	Key Notations	17
6.1	Parameters used in experiments	28
6.2	Median cost of each network for each costing function	29
7.1	Results for PageRank costing function with Linear Threshold and Independent Cascade models	35
7.2	Degree heuristic results	37
7.3	Single Degree heuristic results	37
A.1	Degree heuristic results	44
A.2	Degree heuristic results	44
A.3	Degree heuristic results	45

List of Algorithms

4.1	Highest $\text{Cost}(G = (V,E), b, c)$	18
4.2	Lowest $\text{Cost}(G = (V,E), b, c)$	18
4.3	Average $\text{Cost}(G = (V,E), b, c)$	19
4.4	$\text{PICR}(G = (V,E), b, c)$	20

1 Introduction

There has been a growing increase in the use of large-scale social networks over the last two decades, with recent examples include Facebook and Twitter. These easy to access networks have simplified the task of spreading influence throughout areas previously unreachable. Online marketing makes excellent use of these networks to spread ideas amongst potential customers relating to new products through the use of “word-of-mouth” or viral marketing strategies. It is essential to examine how new ideas are adopted within a social network and how this adoption between individuals causes a propagation effect. The study of network diffusion is not a new concept but the field of research in which it is explored has changed considerably, with some of the first work on influence spread taking place in the medical field studying the adoption of new drugs between medical professionals [15].

Some individuals will be more effective at disseminating information than others within a social network. The influence maximisation (IM) problem was introduced by Domingos and Richardson [10] to determine the smallest subset of individuals in a social network that influence the largest percentage of the network in viral marketing campaigns. The influence maximisation problem was first formalised by Kempe et al as an optimisation problem where the network was defined as a graph G with vertices V representing individuals and edges E indicating some sort of connection between individuals [19]. Given a graph G and an integer k , the influence maximisation problem aims to find a set of nodes S , known as the seed set, that achieves maximal influence spread among all size- k seed sets. In this problem, all nodes are assumed to have uniform cost whereas in reality nodes in social networks usually have different costs - adding a more influential node will be more expensive. Finding the optimal set of nodes to achieve maximal influence spread was shown to be NP-Hard [19].

This project considers a generalised version of the influence maximisation problem known as the budgeted influence maximisation problem (BIM). The difference here is each node is associated with a cost, and a budget b is taken as an input instead of the integer k . The aim is to find the seed set that maximises influence spread where the sum of node costs in the seed set does not exceed the budget. Since the IM problem is NP-hard, it is easy to verify that the generalised BIM problem is also NP-hard. Hence, a heuristic approach must be taken to reduce computational complexity, however, such heuristics provide no guaranteed optimal answer.

The aim of this project is to develop a test suite with which to measure the ability of heuristics to solve the budgeted influence maximisation problem and to develop novel heuristics that outperform previous researched approaches. To do this the test suite must first be created to simulate influence spread through a network given a seed set, then new heuristics need to be researched and tested using the test suite to determine their effectiveness against existing heuristics.

Results from this paper show that the novel approaches taken that utilise cost are more effective at picking seed nodes than previously developed heuristics. Lowest Cost and the potential influence cost ratio heuristics obtained the best results over all test scenarios. Additionally, the use of three costing functions allowed for the generalised performance of heuristics to be measured which eliminates the possibility of heuristics being designed for specific methods of costing.

The remainder of the paper is structured as follows. Chapter 2 discusses previous research relating to the BIM problem and more specifically, previous greedy and heuristic approaches. Chapter 3 details the project management techniques employed, with reference to timelines, tools, and risks. Chapter 4 outlines the design of all components

of the test suite including networks, diffusion models, heuristics, and cost functions used. Additionally, designs and justifications are given for the novel heuristic approaches in this project. In Chapter 5 the implementation is discussed, specifying how the test suite was implemented and issues that occurred during development. Chapter 6 defines the experiment set up used in this project, including the model parameters used and justification of design choices. Chapter 7 explores the results gained from the experiments, both examining expected and unexpected outcomes. Reasons for these results are then discussed and observations are made. The conclusion of the project is presented in Chapter 8 and the evaluation is detailed in Chapter 9.

2 Literature Review

The ability to effectively approximate the optimal solution in the influence maximisation problem is key in many areas of computer science and marketing. A vast amount of research has been undertaken to identify this optimal solution using a number of different approaches. Within this chapter, the influence maximisation and budgeted influence maximisation problem are discussed, in addition to approaches used to gain insight into an individual's influential ability in a large-scale social network. To discuss the influence maximisation problem, the underlying concept of social networks as well as how diffusion models have been constructed to simulate influence spread through a social network need to be examined. Simulations allow evaluations to be made on the effectiveness of information dissemination caused by a specific seed set and can be used to compare performance. In budgeted influence maximisation where the cost of adding nodes to the seed set is not uniform, costing functions must be defined to provide a cost for each node. Functions using random and degree from previous research are examined and the potential use of the PageRank algorithm is also discussed.

To understand each component of the budgeted influence maximisation problem, we must explore the work done in networks, diffusion models, both uniform and non-uniform cost heuristics, and costing functions.

2.1 Networks

Social networks had been researched extensively before the development of online social networks, with anthropologists and sociologists using social networks to better explain societal behaviour [25]. The “social network” was a concept introduced by Barnes [5] in 1954 while researching the social class structure of a Norwegian island parish, discovering that relations between people can be mapped as a network of social interactions. He could then use this mapping to compare the differences between this small island's network to a modern civilised society. The development of the social network from this research fuelled a large amount of development, expanding on his idea into different fields and uses.

Social networks began being used in fields outside of sociology such as economic and family settings [18, 17]. One of the most important applications of social networks is the ability to utilise them to examine the dissemination of ideas. Coleman et al. explore how the adoption of a new drug occurs by mapping the interactions between medical professionals, showing how recommendations between doctors can increase the rate of adoption. Two patterns of diffusion were discovered, the first was a very rapid adoption followed by a quick reversal of adoption. The second was a much slower and steadier increase in adoption. Although allowing a generalised idea into how ideas disseminate, these observations are hard to model in practice due to their seeming randomness, particularly in reversal of adoption, in addition to the time-awareness associated with these models.

Knowing how an idea will disseminate is particularly relevant in marketing and specifically viral marketing [2]. Viral marketing is a strategy where existing social connections are exploited with the aim of encouraging customers to share product information with their friends [22]. Social networks, due to their nature of modelling social connections, can be utilised by marketers to identify key consumers who are most likely to spread the product information to their connections, allowing individuals to be marketed to without direct communication.

Richardson and Domingos [27, 10] employ data mining techniques to viral marketing where ideas are spread through “word-of-mouth” advertising to promote products. When

consumers are targeted through direct marketing, it is crucial to not only consider the value of the consumer but also the influence that they would have on the rest of the network, described as “network value”. The overall value a consumer will have can then be considered when making the decision of whether marketing to this specific person is cost-effective. Here, directed marketing is the main focus with the question of cost-effectiveness the main emphasis. This work serves as a basis of defining the influence maximisation problem but the issue of determining “network value” in an accurate way is unsolved. This project is focused on the viral marketing strategy where we specifically focus on the spread of an idea whereas Richardson and Domingos used a direct marketing approach, only considering spread when determining cost-effectiveness of advertising to an individual.

2.2 Influence Maximisation

The influence maximisation problem was also first introduced by Richardson and Domingos [10, 27] to model “word-of-mouth” marketing strategies. Using probabilistic models they were able to advertise to the individuals that were most likely to spread influence for their viral marketing campaign. This was compared to two additional strategies: no marketing and directed marketing. This concept is used in viral marketing campaigns to achieve cost-effective marketing that reaches as many people as possible. Viral marketing’s aim is to select a subset of potential customers to influence that subsequently influence other consumers to spread awareness of the product to the largest possible number of people.

The influence maximisation problem requires a small set of individuals in a social network to be initially selected to adopt a new idea, usually called being influenced or infected, with the aim of diffusing maximally throughout the network.

The influence maximisation problem is NP-Hard, as the optimal initially influenced individuals, referred to as the seed set, cannot be calculated [19]. The significance of this problem has generated the need for a number of methods for selecting a seed set that causes maximum diffusion. Optimal seed set approximations can be obtained through greedy approaches, or heuristics that utilise network properties and structures to select the seed set as well as many different methods for optimal seed set estimation such as linear programming, however, these are outside the scope of the project.

2.3 Influence Models

Modelling how influence propagates in a social network poses a challenge between realism and computational feasibility. The aim is to emulate real-life as closely as possible to keep the model accurate to what would happen in a real example but there are too many external factors to fully achieve this. Three models frequently used to simulate influence spread are considered by Kempe et al [19]: Linear Threshold, Independent Cascade, and Weighted Cascade. All three use the same underlying framework where once a node has been influenced and becomes active, it cannot return back to an inactive state. This is an assumption that does not hold in real-world scenarios due to an individual being allowed to change their mind but is a simplification that is necessary for these models. Additionally, we are assuming only a single idea is being spread through the network, ensuring that once a node is influenced by the idea it cannot be subsequently influenced by a secondary idea.

2.3.1 Linear Threshold (LT)

The first proposal of a threshold model was made by Granovetter in the context of sociology where a proportion of others must make a decision before the individual does [12]. This developed into the Linear Threshold model where influence is characterised by edges between nodes. Each node v is given a threshold $[0,1]$, which represents the weighted proportion of v 's neighbours required to be influenced before v itself adopts the idea. The model unfolds in deterministic discrete steps given a set of active nodes, with all influenced nodes at time $t - 1$ remaining active and activating any new nodes where the total weight of their neighbours is at least the threshold [19, 9].

2.3.2 Independent Cascade (IC)

Independent Cascade, proposed by Kempe et al., uses a probabilistic approach to influence diffusion and therefore is non-deterministic. Unlike LT where node activation requires a large proportion of neighbours to already be influenced, IC can spread influence to a neighbour with only a single edge.

The model works as follows, let A_i be the set of vertices activated in the i -th iteration and $A_0 = S$. If $u \in A_i$, for any $\overline{uv} \in E$ where v is not yet activated, v is activated by u with an independent probability p , which is known as the propagation probability. Therefore, if l neighbours of v are in A_i , then $v \in A_{i+1}$ with probability $1 - (1 - p)^l$. Once propagation has been attempted through an edge $\overline{uv}$, either from u to v or from v to u , no further attempts are made in subsequent iterations.

Fixed activation probabilities create an issue where higher degree nodes have a greater chance of both directly influencing and being influenced by a larger number of neighbours. The weighted cascade model attempts to deal with this issue.

2.3.3 Weighted Cascade (WC)

The Weighted Cascade model solves the issue of bias towards high degree nodes by calculating the activation probability between active node u and inactive node v as

$$p_{uv} = \frac{1}{\text{degree}_v}.$$

The IC and WC models first introduced by Kempe et al. are computationally expensive to run. To improve this Chen et al proposed an improvement that reduced the runtime from $O(knRm)$ to $O(m)$ and $O(mn)$ for the IC and WC models respectively, where k is size of initial seed set, n is the number of nodes, m is the number of edges, and R is the number of iterations, by only considering a single hop between active to inactive node in a single iteration.

All three models provide a method that is partially related to real-world scenarios, with Linear Threshold closely following a peer pressure approach and IC and WC focusing more on the random aspect of influence spread. The simplicity of their design allows for faster computation when compared to more complex variants of these models which is important as random processes require many repetitions to ensure an accurate average. The disadvantage of this being loss of realism due to simplicity of models compared to the complexity of real-world scenarios.

2.4 Uniform Cost

The aim of the influence maximisation problem is to pick a starting set, or seed set, of nodes such that an idea is maximally diffused. Finding this optimal seed set is NP-Hard

and approaches used vary greatly from greedy methods to heuristics.

Uniform cost assumes that nodes added to a seed set have identical cost and therefore a seed set is constrained by a certain number of nodes, also called a seed set size. This differs to the non-uniform cost space where each node has a distinct cost associated to it. Instead of a defined size for the seed set, a budget is specified where nodes are added providing there is space remaining in the budget.

2.4.1 Greedy

Kempe et al. were the first to propose a greedy approach to finding the optimal seed set. They took a hill-climb approach with each iteration adding the node that causes the maximum marginal gain, enabling it to obtain $1 - \frac{1}{e}$ approximation ratio. Although this approach outperforms more primitive degree and centrality methods, computational complexity issues result in unsuitability for large scale social networks [19].

An improved greedy methodology was developed by Leskovec et al. called “Cost-Effective Lazy Forward” (CELFG) [23] which utilises the sub-modularity property of the influence maximisation problem. This allows a reduction to the number of evaluations on the influence spread of vertices, leading to a speed-up of up to 700 times compared to the original greedy algorithm [19, 9, 23]. Goyal et al. utilised these optimisations to improve the CELFG algorithm and propose CELFG++, using Monte Carlo simulations to increase efficiency by 30% - 35% [11].

Despite the optimisation made to the greedy approaches, the issue of computational complexity still persists making them unsuitable for use in large-scale social networks. A different set of techniques involve the use of heuristics taken from the social network. Heuristic approaches are less computationally complex which allows them to be used effectively in real-world domains with large-scale social networks such as viral marketing. The disadvantage is their success in obtaining the optimal seed set is significantly reduced compared to greedy approaches.

2.4.2 Heuristics

The heuristic most commonly used for seed set selection is degree. Degree is heavily utilised due to the assumed correlation between the number of neighbours a node has and the amount of influence the node possesses within the network [31]. Consequently, higher degree nodes are more likely to be selected for the seed set.

Chen et al. proposed two new heuristics: Single Discount and the improved version Degree Discount. Single Discount considers the situation where node u is selected as a seed using a degree heuristic. Its neighbour v should reduce its degree by one to account for it having one less node to influence if selected as a seed. The improvement, Degree Discount, considers that node v becomes active with a probability p , in addition to degree reduction shown in Single Discount. Degree is discounted taking into account the propagation probability and the number of active neighbours. Results showed much greater influence spread than degree and centrality-based heuristics and achieves almost matching influence with CELFG. Run-time compared to CELFG is 34% less, completing in only a few milliseconds [9]. Degree Discount provides a balance between run-time and finding a seed set that causes large influence spread while also working effectively for the LT and IC models.

Additionally, Chen et al. devised two approximation algorithms, PMIA and LDAG to solve the IM problem under the IC and LT models respectively. Although effective in

their respective domains, the specific use-case of these heuristics reduces their usefulness in a more general case such as the BIM problem.

The comparison of the greedy to heuristic approaches illuminates the trade-off of speed to optimal seed set approximation that presents itself within the influence maximisation problem. While smaller networks are feasible for greedy algorithms, once size is increased heuristic approaches become a more viable option.

Up until now, the heuristics discussed are only focused on the problem of influence maximisation where the cost of selecting a node as a seed is uniform. Their design being centred around uniform cost means that they are unsuitable for problems involving budgets and non-uniform costs. New adaptations of greedy and heuristic approaches need to be devised to solve the generalised problem of budgeted influence maximisation.

2.5 Non-Uniform

Brief mentions of cost of node selection not being uniform are made in early work. Domingos and Richardson define the influence maximisation problem for practical applications in marketing. Their aim is to optimize the amount of marketing funds spent on each customer to ensure a net gain, or profit, from each customer [27]. Since marketing funds are finite, we can assume they are working within a budget and with customers having different monetary value, we have the bases of the budgeted influence maximisation problem. BIM offers a problem that is much more applicable to real-world marketing scenarios with companies wanting to maximise the spread of their new product within a monetary budget.

The generalised budgeted influence maximisation problem can be framed and optimised similarly to the simpler influence maximisation problem. Khuller et al [20] present a $1 - \frac{1}{e}$ approximate solution which requires enumerating all the size-3 seed set and apply the greedy algorithm to select the remaining nodes. An improvement is made to this approximation factor utilising the enumeration technique which allows a performance guarantee of $1 - \frac{1}{e} + \epsilon$. Krause and Guestrin [21] show that this approximation can be further improved to $1 - \frac{1}{\sqrt{e}}$ with slight changes made to the greedy algorithm.

Greedy approaches to the BIM problem suffer from the same obstacles that are present in the IM problem relating the computation complexity. However, with the additional aspect of budgets, nodes that cannot fit within the budget can be discounted which leads to marginally improved performance. However, this is unable to reduce run-time and complexity adequately to be practical on large-scale social networks.

Expanding on work by Khuller et al. Nguyen and Zheng [26] propose an approximation algorithm (called BIM) based on a greedy algorithm for Budgeted Maximum Coverage [20], in addition to two other heuristics. Their algorithms achieved better results when compared to both CELF and PMIA on the Weighted Cascade and Trivalency models. However, they only investigate randomly costing the nodes which is not an accurate representation of real-world marketing settings.

Han et al. propose a further three heuristics, sectioning nodes into roles dependent on their degree characteristics [16]. The first is a strategy that chooses the most influential nodes in the network as seeds called “Billboard”. The second, “Handbill”, selects the most cost effective nodes by looking for low-degree nodes with high-degree neighbours that will allow the idea to spread. The “Handbill” approach is only successful in cases where the cost is linked to the degree as different costing methods might cost these low degree nodes highly. Additionally, searching for nodes with high degree neighbours is

only successful in spreading influence if the initial low degree node manages to successfully influence its neighbour, which is not guaranteed. Combining these is the third strategy where the “best” seeds from the “Billboard” and “Handbill” seeds sets are selected to obtain a composite solution. The combination algorithm, although allowed marginal gain, increased computational complexity considerably. The need to execute both “Billboard” and “Handbill” in addition to simulated annealing to combine the two into a “super” seed set causes an increase of up to three times compared to “Handbill”.

A linear programming approach taken by Güney [13] where the objective is identifying influential individuals to maximise influence given a limited budget. An integer program is developed that approximates the original problem and solves by a sample average approximation scheme. Experimentation indicated that the SAA methodology provides better results than the greedy method without increasing computational complexity when the network is small. As the network size increases, the solution time to solve the linear program grows fast leading to drastically decreased performance and the greedy method starts to outperform.

2.6 Costing

Costing functions are utilised in budgeted influence maximisation to determine the cost associated with adding a node as a seed. Costing is important as it determines the behaviour a BIM algorithm should take to achieve the optimal seed set. Additionally, it allows relation to real-world marketing scenarios if a cost function is designed similarly to how the everyday world works.

Work previously discussed by Nguyen and Zheng [26] only used random node costing when designing their heuristics. Random costing is not related to real-world scenarios as the price of marketing a product to a potential customer is not random in nature. Instead, it is likely correlated to the amount of influence an individual has.

Costing nodes by degree can be a method to tackle this, Han et al. utilise this by defining the cost of a node to be $outD * p + 1$, where $outD$ is the out-degree of a node in a directed graph and p is the propagation probability [16]. Since a nodes with higher potential influence spread costs more, this more closely relates to how advertising is conducted in the real-world. However, within this research, degree is the only cost function used which allows their heuristics to be optimised for solving only degree based costing scenarios.

Arthur et al. propose a pricing method a seller could utilise when implementing a viral marketing strategy [4]. In addition to influence from neighbours, each potential customer is assigned a cost by the seller, indicating the price a consumer needs to pay in order to buy the product. The individual can then choose to buy or not buy the product at the given price, this price is then changed after each time step. In this strategy the seller gives incentives to buyers in the form of discounts or cashback to secure sales. The BIM problem is different as each individual is not offered different prices. However, this strategy creates a new perspective to the problem where the seller is defining the costing associated with each individual in the network.

A potentially better strategy of costing nodes based on the amount of influence it has, is the method used by Google to rank their search results. PageRank is a way of measuring the importance of website pages by analysing the links between pages [6]. This method can be used in social networks with nodes being webpages and links being social connections to identify nodes with the greatest influence [30]. It does not relying simply on a single number to determine potential influence, allowing it to produce different results to random and degree costings. When costing nodes it is important to provide a

number of different methods of costing to allow for generalisation in approaches to solving the BIM problem. As of yet, research into the BIM problem utilises only random or degree oriented costing functions leaving potential for PageRank to provide a new approach to costing.

Previous research has investigated a number of key areas in both the influence maximisation and budgeted influence maximisation problems particularly focusing on greedy and heuristic approaches to the problem. A significant focus is on utilising degree to pick seed nodes due to the assumption that a high degree node is also highly influential, a concept heavily employed in work by Chen et al [9, 8] for the IM problem and Han et al [14] with their heuristics for solving the BIM problem. However, current literature in the BIM problem do not consider different approaches to heuristics that fully utilise the new network information provided by the problem - the costing. Additionally, costing functions used in BIM research are limited to two main methods, random and degree, which don't allow for enough generality when reviewing heuristic performance.

3 Project Management

The use of project management techniques was key to the success of the project due to the number of components involved in the development. In this chapter the methodology and approaches taken during the project are outlined, with specific reference to timelines, tools, and risk management.

3.1 Methodology

Due to the project's component nature, an agile approach was taken in the development. Each component could be implemented and tested during a single agile sprint allowing increasing functionality to be built up over successive sprints, this approach is much more suitable than a more traditional waterfall method. The systems can then be seamlessly integrated and only minor adjustments relating to interfacing will need to be made.

The four main constituent parts of the development were: networks, influence models, heuristics, and cost functions. There was a specific order to development that was followed due to the underlying mechanics of how the whole system operated. For example, the heuristics could not be tested until after networks and influence models, as measuring influence spread produced from seed sets by these heuristics is an element of testing.

These distinct sections created easily identifiable milestones and allowed easy distinguishing between agile sprints. Although it reduces flexibility of development, the inclusion of these milestones allows more structure to the plan and allows for ease of timeline creation. Additionally, this structure is only observed on a high level and flexibility is still offered both within each constituent part and during the integration phase.

A feature of agile strategies is frequent communication to ensure there is a consensus on project requirements. Weekly supervisor meetings were undertaken, allowing the progress and direction of the project to be monitored. Each week, the progress and planned work were discussed.

3.2 Timeline

Although the project has a natural structure, more specific deadlines needed to be defined to ensure effective progression. These deadlines were only considered guidelines and amendments could be made to ensure progress occurred efficiently.

The project was divided into five phases, each representing a different distinct stage in the project. Additionally, there were four official deadlines that could not be changed: the project proposal, presentation, interim report, and final dissertation. Deadlines within the timeline took into account these official deadlines when first being defined to prevent interference.

Phase one was dedicated to research into previous work within influence maximisation and more specifically, work within budgeted influence maximisation. Planning of the project was also carried out to define work required in each subsequent phase in addition to allowing the project proposal to be written. Phase two was the development phase with all four development components being given greater specificity to allow tracking of progress to be more efficient. Development schedules are challenging to estimate because of unforeseen difficulties that need to be overcome. To account for this, the timeline allotted contingency time to ensure the project remains on schedule even if problems appear. Phases three and four were heuristic research and development respectively. The iterative process of researching and implementing required that these phases be grouped together to allow moving between research and development without crossing big milestone boundaries. The final phase involves testing all the heuristics including newly developed

approaches and the final dissertation write-up. Testing being completed simultaneously with the write-up allowed the results to be in recent memory during the writing of the final dissertation. Additionally, previous research suggested that the testing was likely to take considerable time and completing the write-up during this time would make efficient use of time. The GANTT chart in Figure 3.1 shows the original timeline created.

From Figure 3.1, testing was allotted time in the final phase of the project just before the write-up, and no testing was done prior to this. Early into the project, in the development phase, it was found that leaving testing until the end of the project was not ideal. Undertaking the testing within the development phase was considerably more efficient for a number of reasons. Firstly, it allowed overall testing and gathering of results to be carried out in the place of simple testing of heuristics. This allowed testing to have more of a purpose than just confirming that everything worked as expected and instead there was an end goal of gathering usable results. Secondly, uniform cost does not require costing functions and therefore the testing could be accomplished before their implementation. Testing in this way would allow us to save time owing to staggering the testing and allowing development to continue alongside testing. Thirdly, results being gathered prior to phase three means the results that a new heuristic needs to outperform are already known, allowing a more efficient heuristic development phase.

The updated timeline is displayed in Figure 3.2, testing is moved into phase two in addition to renaming of phases three and four objectives for clarity.

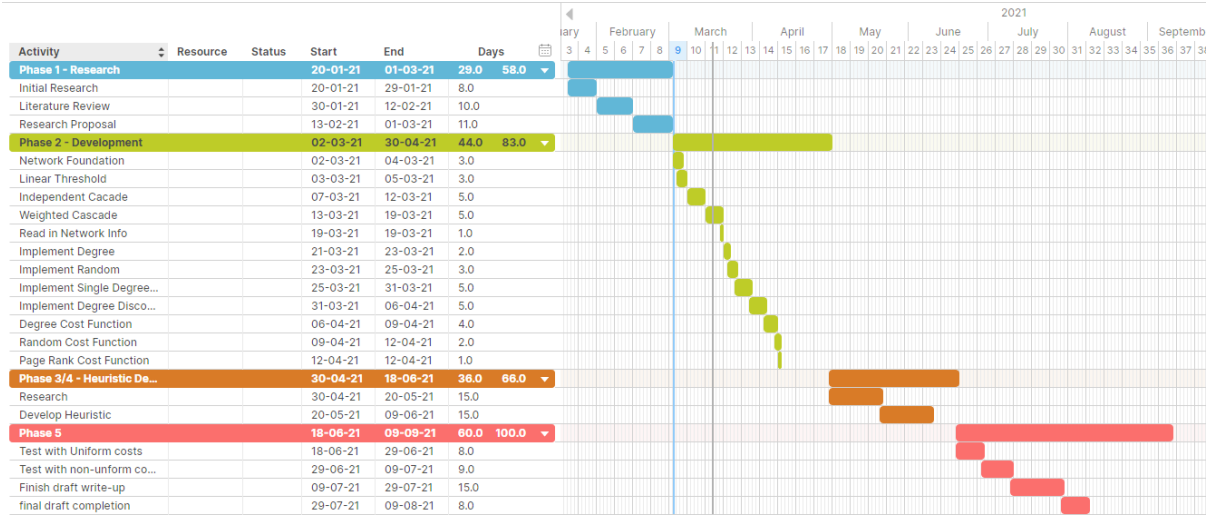


Figure 3.1: GANTT chart of project timeline showing time and duration of each phase.

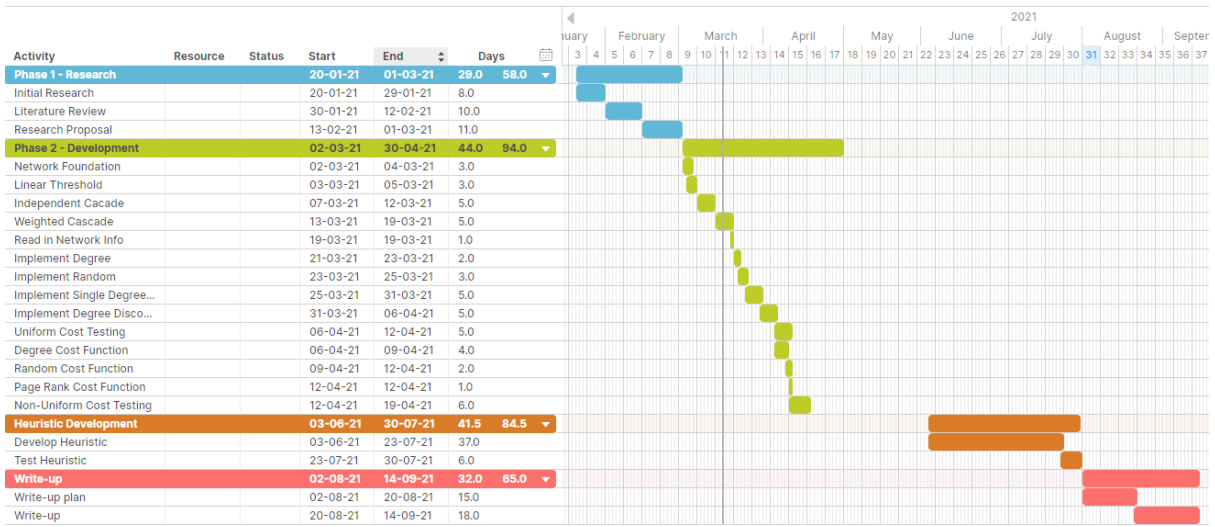


Figure 3.2: Updated GANTT chart of project timeline with testing moved to phase two.

3.3 Tools

The entirety of the development was carried out using Python due to the ease of use in addition to the libraries available within the language. This project required significant development and the speed of development that Python offers helped considerably to meet the time constraint of this project.

3.3.1 NetworkX

NetworkX is a Python package for the creation and manipulation of complex networks [14]. The functionality it provides was paramount to the success of the project due to being able to utilise already implemented features to ensure efficiency. Moreover, NetworkX provides intuitive interfacing with other Python libraries used in this project which was a primary factor for the choice of underlying programming language. Additionally, it provides easy access to network properties such as degree - a property heavily utilised in influence maximisation heuristics, and conversion between directed and undirected graph

which is useful because although LT and IC use undirected graph, WC requires the graph to be directed.

3.3.2 Network Diffusion Library

In addition to NetworkX, a crucial library was the recently developed Network Diffusion library (NDlib) [28, 29]. The library provides implementations of many different network diffusion models together with the ability to visualise the results. To run a model, a seed set of initially active nodes in addition to a NetworkX graph are passed and results on influence spread are returned. Use of NDlib allows influence models to be efficiently build, increasing allotted time for more complex sections of the development such as phases three and four.

3.3.3 Python Libraries

In addition to NetworkX and NDlib, many other Python libraries were utilised. Matplotlib was used extensively to display results for testing and comparison purposes. The random library was also used heavily for heuristics such as random and for the IC and WC models that have probabilistic elements.

3.3.4 SNAP

SNAP is a Python library for manipulation and analysis of large networks developed by Stanford University [24]. The library itself was not utilised, however, it offers a large network dataset collection for use in research. The GitHub social network and Amazon co-purchasing network used in this project were obtained through this collection. Using these networks provides the ability to investigate the budgeted influence maximisation problem without the need to generate synthetic social networks. Without these datasets, gaining real-world social network data would require scraping data from social networks which would be extremely time consuming.

3.3.5 Git

The use of Git provided version control to the project and due to the research element it was important to be able to branch to explore new ideas. Git was crucial to the project as a method of ensuring an up-to-date backup of the work was available at all times.

3.4 Risk Management

A number of potential risks were associated with the completion of the project. These include:

1. Delays due to illness
2. Unexpected technical challenges
3. Poor version control

Due to the nature of the project being completed individually, the impact of illness had the ability to temporarily stop work. The probability of this risk was increased further by COVID-19 which could cause significant delays to work. Mitigation to this was specified in the timeline, where time was left between phases to allow for unexpected circumstances that effect the work being done. Additionally, the scope of the project

could be scaled back if a significant delay occurs such as removing individual costing functions or heuristics.

Unexpected technical challenges have a high chance of occurring within the project particularly in the more complex phases. Mitigation was carried out by minimising the replication of existing methods, resulting in more time being spent on the novel aspects of the project. Moreover, the extra time between phases mentioned above could be utilised for this issue if the challenge could not be avoided.

Version control was ensured through the use of Git. Development was carried out using an iterative approach, making version control through Git a key component of the project. Documentation was saved using the University of Warwick OneDrive which provided a backup any time a document was saved. OneDrive together with Git reduced the risk of data loss to extremely low.

3.4.1 Encountered Risks

Over the development of the project two of the three potential risks were encountered. Delays due to illness were experienced due to COVID-19 which caused minor delays, however, the contingencies put in place reduced the effects resulting in minimal disruption to the overall project.

Unexpected technical challenges were also encountered in the development caused by the use of the NDlib. Initial use of the library caused a dramatic decrease in development time due to supplied implementations of diffusion models. However, It was found that these models produced inaccurate result. The solution required rewriting the diffusion models to work as intended, using the NDlib models as a template. The three weeks of contingency preplanned at the end of phase two mitigated the potential deviation from the timeline, resulting in no effect to the overall project.

4 Design

As alluded to in Chapter 2 there has been significant work conducted into the influence maximisation problem but considerably less in the budgeted influence maximisation problem. Previous research utilises random costing functions in addition to using a nodes degree as its cost, causing issues with heuristics which use degree heavily in seed selection. Moreover, budgeted heuristics in earlier work carried out take approaches similar to uniform costing by utilising network properties. In this chapter, the test suite design is outlined including networks, diffusion models, heuristics, and costing functions. Additionally, novel heuristic approaches and a new costing function are discussed.

4.1 Networks

Network selection was critical to the project because the results achieved by the heuristics depends on the social network architectures. There are two main types of social network. Synthetic networks which are artificially created networks designed to contain specific properties with which the heuristics can be tested. Non-synthetic networks are the opposite as they are social networks taken from the real-world such as Twitter or Facebook. Synthetic networks have the benefit of being able to manufacture properties that are difficult to find in real-world networks. Additionally, since networks are created, instead of sourced, no work is needed in performing data collection or parsing a real-world network to make it useable to a computer. However, since it is far detached from a everyday scenario, the results obtained might not hold when applied to practical problems. Non-synthetic networks solve this problem by allowing real-world networks to be used in problems involving networks. The main issue of collecting enough data to have a useful network has been solved by databases of large-scale social networks such as SNAP (Stanford Network Analysis Project) [24] and ICON (The Colorado Index of Complex Networks) [1].

This project focuses on non-synthetic networks for testing heuristics mainly because the budgeted influence maximisation problem has applications within marketing and showing results with real-world networks is more effective to prove this use case. There are many areas of social networks available within these databases and it is important to choose these effectively. Choosing multiple networks from similar environments such as online social networks will degrade the results gained due to not providing a more generalised view of how the heuristics perform.

Taking this into accounts, three networks were selected for this project. An Amazon co-purchasing network, a GitHub social network, and an Arxiv citation network. The node and edge count of each network is displayed in Table 4.1.

Network	Nodes	Edges
Amazon	262,111	1,234,877
GitHub	37,700	289,003
Arxiv	37,154	231,584

Table 4.1: Network node and edge counts

4.2 Models

The models selected are LT, IC, and WC as they provide a variety of influence spread dynamics, allowing for good quality results to be acquired. These models are used

throughout influence maximisation literature, allowing the results from the heuristics to be directly compared against previous solutions.

Linear Threshold is a non-probabilistic model and therefore offers determinism. The benefit of this is the influence spread is easily observable and understandable, making it a good model for identifying how the seed set a heuristic selects affect the overall influence spread. Additionally, the LT model is based on the theory of peer pressure where, if enough neighbours adopt an idea, the individual is pressured into also adopting the idea, regardless of the positive or negative of doing so. This is a phenomenon common in everyday society and therefore can be exploited by viral marketing. Since marketing is a main use of the BIM problem, using this model will show the influence diffusion in this scenario.

Probabilistic models allow for the simulation of randomness associated with influence spread. Since accurately modelling each individuals' potential actions that lead to them spreading influence in a social network is unattainable, introducing randomness is the most computationally efficient method to simulate this as accurately as possible. Independent Cascade is a widespread and effective probabilistic diffusion model, showed through its used in previous work utilised by this project. Weighted Cascade is the third model as it solves the degree bias issue of IC and therefore provides a different approach to influence spread for heuristics to be tested against. By using the IC and WC models, a good understanding of heuristic performance in a probabilistic setting is determined and with the addition of LT, a reliable all-encompassing insight into a heuristic's performance is gained.

4.3 Heuristics

Heuristics development is the major focus of the project as each heuristic defines a certain logic in which to create a seed set that maximises influence. For this project, a number of pre-existing heuristics were developed to serve as a comparison with which to test new heuristics performance against. Selecting these was a key design challenge as a comprehensive range of simple to more complex heuristics is required to allow for good comparisons but since there is only finite time in the project, not every heuristic could be implemented.

Two basic heuristics chosen were random and degree as they are common in previous research as a baseline and are quick to implement, leaving more time for more complex algorithms. Random provides a real baseline as it allows fully random performance to be measured and compared. Degree is a more useful baseline since it comes from an idea that is utilised in previous research, higher degree nodes will spread more influence. Furthermore, it is the basis of the more complex heuristics so allows a comparison of the additional benefit a more complex heuristics gives.

For more complex heuristics there are many options all offering good performance relative to the simple approaches. The Degree Discount heuristics by Chen et al [9] were chosen for this project due to their ability to work with every influence model used. Additionally, the runtime of these heuristics is low when compared to other heuristics while offering similar performance. PMIA and LDAG also proposed by Chen et al. could have been used but they are only useable on the IC and LT models respectively, giving a less generalised approach. These heuristics are also much more complex than Degree Discount and therefore require a considerable amount of time in implementation and testing to ensure they are working correctly. Completing these within the time frame would potentially cause issues and impact other areas of the project and therefore were left out.

The Degree Discount heuristics consist of two degree related approaches. The first is similar to standard degree with a simple discount of each neighbour in the seed set called Single Discount. Single Discount can be utilised in all three models allowing it to give a generalised comparison. The second is Degree Discount which uses a more complex method to discount degree and although this heuristic only works effectively with the LT and IC models, its ability to be used on two models still provides better generality than PMIA and LDAG.

The application of the budgeted influence maximisation problem in marketing is arguably closer to real-world scenarios than the more specific influence maximisation problem. The reach of a marketing campaign is usually defined by the budget available to the company. The main difference between IM and BIM is the costs that are given to nodes which can be seen as additional information about a node. Research into heuristic approaches leverage network properties, however, BIM problem research fails to leverage the new property given by definition of the problem. Three heuristics are defined below that utilise this cost property to select seed nodes. Key notations used in this chapter are summarised in Table 4.2.

G, V, E	the graph, its set of vertices and edges
n	the number of vertices in graph G
m	the number of edges in graph G
S	the selected seed set
tc	the total cost of the current seed set
c_v	the cost of the vertex v
b	the budget cost of the seed set
SV	a set of vertices of graph G sorted in ascending order by cost
d_v	the degree of vertex v
dr_v	the ratio between potential influence and cost for vertex v
t_v	number of neighbours of vertex v already in the seed set

Table 4.2: Key Notations

4.3.1 Highest Cost

The first algorithm proposed uses this concept to focus on adding the highest cost nodes possible. The aim is to understand the effectiveness of choosing the individuals in the network that are defined as the most expensive. In each iteration, the node u with the greatest cost is selected and added to the seed set S only if its cost is less than or equal to the remaining budget. Regardless of whether the node u is added as a seed, it is removed from the set of potential seeds V , allowing a new highest cost node to be selected in the next iteration. This is repeated until the budget has been filled or the vertices set is empty.

The scenario where the highest cost node is greater than the entire budget is possible, giving the possible instance where the seed set is empty. In a marketing setting this is unrealistic as, the marketing campaign would not be abandoned if they were unable to pay the highest cost celebrity to market their product. Removing current highest cost node u from the set of vertices regardless of its addition to the seed set allows continuous iterations until either no nodes remain in V or the total cost of the seed set tc is equal to the budget b . This then ensures the budget is maximally utilised by the algorithm. Highest cost is summarised in Algorithm 4.1.

Algorithm 4.1 Highest Cost($G = (V, E)$, b , c)

```
1:  $S = \emptyset$ 
2:  $tc = 0$ 
3: repeat
4:    $u = \arg \max_v \{c_v | v \in V\}$ 
5:   if  $tc + c_u \leq b$  then
6:      $S = S \cup \{u\}$ 
7:      $tc = tc + c_u$ 
8:   end if
9:    $V = V \setminus \{u\}$ 
10: until  $V = \emptyset$  or  $tc = b$ 
```

4.3.2 Lowest Cost

To provide a comparison against the effectiveness of selecting the highest cost nodes, the opposite is considered. In a similar assumption to highest cost, lowest cost nodes are defined as the individuals in the network that hold the lowest potential influence spread, assuming influence spread is correlated to cost. This could be due to having very limited connections with which to spread influence, or they are positioned poorly within the network such as being in a community of exclusively low degree nodes.

There are two notable differences between finding highest and lowest cost seed sets. The first is straightforward in that instead of finding the node with the highest cost using $\arg \max$, $\arg \min$ is used to find the lowest cost node. The second involves how the terminating condition of the loop is defined on line 10. Previously, the Highest Cost heuristic, if the highest costing node is not within the remaining budget, the algorithm will keep iterating until either of the two conditions are met. With Lowest Cost, once the lowest cost node does not fit within the budget, no subsequent node will fit. The algorithm can then terminate, this is shown in Algorithm 4.2.

Algorithm 4.2 Lowest Cost($G = (V, E)$, b , c)

```
1:  $S = \emptyset$ 
2:  $tc = 0$ 
3: repeat
4:    $u = \arg \min_v \{c_v | v \in V\}$ 
5:   if  $(tc + c_u \leq b)$  then
6:      $S = S \cup \{u\}$ 
7:   end if
8:    $tc = tc + c_u$ 
9:    $V = V \setminus \{u\}$ 
10: until  $tc \geq b$ 
```

4.3.3 Average Cost

Combining the Highest Cost and Lowest Cost heuristics, a middle ground between the two previous approaches is defined. Since cost is being used as a metric of potential influence in these algorithms, the average individual in a network has a better ability to spread influence than the lowest costing node. Average costing nodes come with a higher cost to reflect this but their cost is significantly less than that of the highest costing nodes.

The average cost node is defined as the median node in the set of all potential seed nodes sorted by cost. A common network property of large-scale social networks is being scale-free, meaning a small subset of nodes hold very high degrees whereas the large

majority have small degrees [7]. These high degree nodes cause the mean to be inflated, making the median the only accurate average measurement.

The set of vertices is first sorted with respect to each node's cost, allowing us to find the median. Similarly, to the previous algorithms, a check is made for whether the node cost is not greater than the remaining budget and added to the seed set accordingly. Iterations are made until either the budget is full, or the set of potential seed nodes is empty. The algorithm is defined in full in Algorithm 4.3.

Whereas finding highest and lowest cost are at worse $O(n)$, with n being the number of nodes in the graph, the task of sorting is required for Average Cost. This causes the complexity to increase to $O(n \log n)$, decreasing its efficiency on very large networks with large budgets. However, this effect is negligible and will only become notable in scenarios that are highly unlikely.

Algorithm 4.3 Average Cost($G = (V, E)$, b , c)

```

1:  $S = \emptyset$ 
2:  $tc = 0$ 
3:  $SC = \text{sort vertices } V \text{ by cost } c$ 
4: repeat
5:    $u = \text{median vertex of } SC$ 
6:   if  $(tc + c_u \leq b)$  then
7:      $S = S \cup \{u\}$ 
8:      $SC = SC \setminus \{u\}$ 
9:   end if
10:   $tc = tc + c_u$ 
11: until  $SC = \emptyset$  or  $tc \geq b$ 

```

4.3.4 Potential Influence Cost Ratio

Degree Discount, proposed by Chen et al [9], is a heuristic discussed in Chapter 2 that involves discounting the degree of a node based on the number of its neighbours already added to the seed set. Inspiration is taken from this idea in the definition of Potential Influence Cost Ratio (PICR); however, modifications are made both for increased performance and optimisation for the BIM problem.

For the IC model, a fixed propagation probability p is defined across the entire model. This is different from WC where the probability is dependent on the degree of the node. If we assume the example where node u has been selected as a seed, then with probability p its neighbour v will be influenced and in that case it does not need to be added as a seed.

In the PICR algorithm, the ability for influence to make multiple hops in a single iteration of the model is assumed to be impossible, allowing the calculations in the algorithm to be computationally manageable. The neighbourhood of a node u is defined as a sub-graph containing the node itself in addition to all nodes that share an edge with the node u , defined by $N(u) = \{u\} \cup \{v \in V \mid uv \in E\}$. Additionally, we let t_u be the number of neighbours of vertex u that are already in the seed set.

From the neighbourhood of a node u , we can denote the following formula: $(1 + ((d_u - t_u)p))$. The potential influence of a node u is defined by the number of remaining neighbours that are not already in the seed set $(d_u - t_u)$ multiplied by the propagation probability p that each neighbour will be activated, defined in Equation 4.1. Lastly, we have a guaranteed increase in influence spread of 1 by definition of adding u as a seed, giving Equation 4.2. This theorem allows us to estimate the influence associated with adding a node to the seed set without needing to explicitly run simulations and although

is not as accurate as simulations, it has considerable efficiency benefits.

Utilising this to select seeds may be sufficient for the influence maximisation problem but the generalised budgeted influence maximisation problem requires an additional step. By dividing the potential influence of a node by its cost, new knowledge is gained into the influence potential of a node in relation to its cost. The nodes that give the best “cost-effectiveness” can then be identified and added as seeds, maximising the theoretical influence gain for the cost of the node. Giving the equation shown in Equation 4.3.

$$(d_v - t_v)p \quad (4.1)$$

$$(1 + ((d_v - t_v)p)) \quad (4.2)$$

$$(1 + ((d_v - t_v)p))/C_v \quad (4.3)$$

This algorithm provides hugely better generality than other previous solutions as it has the ability to work effectively with any costing function compared to proposed solutions specifically designed for one type of costing. Algorithm 4.4 implements the Potential Influence Cost Ratio heuristic. The algorithm is $O(n \log n + m)$ with m denoting the number of edges, giving it only marginally worse performance than the average cost heuristics.

Algorithm 4.4 PICR($G = (V, E)$, b , c)

```

1:  $S = \emptyset$ 
2:  $tc = 0$ 
3: for each vertex  $v \in V$  do
4:   compute its degree  $d_v$ 
5:    $dr_v = (1 + (d_v p))/c_v$ 
6:    $t_v = 0$ 
7: end for
8: repeat
9:    $u = \arg \max_v \{dr_v | v \in V\}$ 
10:  if  $(tc + c_u) \leq b$  then
11:     $S = S \cup \{u\}$ 
12:    for each neighbour  $v$  of  $u$  where  $v \in V$  do
13:       $t_v = t_v + 1$ 
14:       $dr_v = (1 + ((d_v - t_v)p))/c_v$ 
15:    end for
16:  end if
17:   $V = V \setminus \{u\}$ 
18: until  $SC = \emptyset$  or  $tc \geq b$ 

```

The use of probability within the calculation of potential influence provides a more accurate result since the probability associated with each node’s edge is taken into account. However, this approach is only effective with probabilistic models as influence within non-probabilistic models are not associated with probability. The PICR heuristic is only usable on the IC model because of its fixed probability nature, additionally, previous research has shown the heuristics that include probability obtain good results from the LT model [9].

To combat this a further adaption to PICR is proposed that removes the probability from the potential influence cost ratio calculation named PICRnp. This new heuristic can be used on all three models without issue and therefore allows for increased generality.

4.4 Cost Functions

Cost functions define how the nodes in the network are costing when solving the budgeted influence maximisation problem. Cost functions are constrained by the limited network information available for deriving a cost. Random and degree are the two main approaches taken in previous work and this project will utilise these.

Random costing is important as it creates the unique scenario where the cost of a node has no correlation to the influence a node could diffuse. Although, this is not realistic to real-world events, it tests heuristics to maximise influence with cost being completely arbitrary but without being uniform. Therefore, it is important to include as it covers a case that is untestable otherwise.

The degree cost function is the opposite as the cost is highly linked to the influence a node could have, given the assumption that higher degree relates to higher influence spread. Degree provides a vastly different scenario to the random costing and presents a heuristic with an almost completely different challenge particularly as degree is a key property used for seed set selection in many heuristics particularly in this project.

4.4.1 PageRank

PageRank is an algorithm used by Google to rank their search engine results and is a method of measuring the importance of webpages based on hyperlinks between pages. The approach is used to indicate the probability that a person randomly clicking on links will end up on a specific webpage. The internet is denoted as the webgraph, describing the direct links between pages. It is a graph that consists of vertices corresponding to pages on the internet, and edges denoting hyperlinks that exist between pages. The webgraph is the format that is used to compute PageRank and therefore is in essence an algorithm for identifying important nodes within a graph.

Initially, every node is given a uniform initial value such that the sum of all values is 1. In each iteration, each node will equally divide its value between all its neighbour that it shares an edge with. Consider the following example, we have nodes A, B, C, and D. Node A has links to B and C, node B has links to A, C, and D, node C has a link to D, and node D has a link to A. The initial values will be $\frac{1}{N}$ where N is the number of nodes, giving the value $\frac{1}{4}$ to each node. Node A will half its existing value, $\frac{1}{8}$, between nodes B and C, node B will split its value three ways, $\frac{1}{12}$, between nodes A, C, and D, node C will send all its value to D, and node D will send all its value to A. After this iteration, node A, B, C, D have values $\frac{1}{3}, \frac{3}{8}, \frac{5}{24}, \frac{1}{3}$ respectively. This example is seen in graphical form in Figure 4.1.

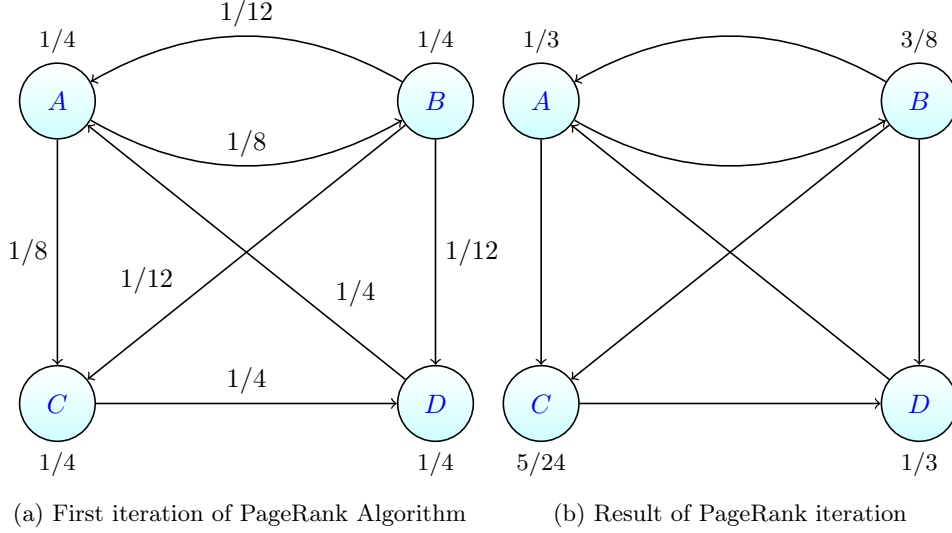


Figure 4.1: Example of PageRank Algorithm

Additionally, since the individual randomly clicking links must eventually stop, a damping factor d , usually set to 0.85, is defined which is the probability at any step that the person will continue clicking. It is subtracted from 1 and divided by the number of nodes N , then it is added to the product of the damping factor [6] and the incoming PageRank score of a node.

$$PR(A) = \frac{1-d}{N} + d \left(\frac{PR(B)}{L(B)} + \frac{PR(C)}{L(C)} + \frac{PR(D)}{L(D)} + \dots \right) \quad (4.4)$$

Equation 4.4 is defined with $PR(B)$ being the PageRank of node B and $L(B)$ being the number of outbound links for node B, assuming the node A has at least one inbound link from node B.

The algorithm can be defined iteratively with the initial distribution at time $t = 0$ as Equation 4.5.

$$PR(p_i; 0) = \frac{1}{N} \quad (4.5)$$

With N being the total number of nodes and $p_i; 0$ being the PageRank value of node i at time 0.

At each time step, the calculation is defined in 4.6.

$$PR(p_i; t+1) = \frac{1-d}{N} + d \sum_{p_j \in M(p_i)} \frac{PR(p_j; t)}{L(p_j)} \quad (4.6)$$

Using this notion, PageRank can be applied to any graph to gain knowledge on node importance. In a social network, it gives the ability to define node costs without the use of a single network property such as degree, but allows costings to be based on node importance. Higher importance nodes are inherently more well connected and therefore

more likely to spread influence, giving a costing function that relates more to the real-world where cost originates from more than just the number of connections a node as, in the case of degree costing.

The overall design of the project is tailored to ensure there a large number of conditions with which to test heuristics against. This is particularly key with the cost functions as adding additional costing methods contributes greatly to testing the generality of a heuristic. Additionally, the networks chosen are picked to compound this focus on generality by providing large-scale social networks from vastly different environments.

5 Implementation

5.1 Networks

The three non-synthetic social networks for this project were taken from two separate sources. The Amazon co-purchasing and GitHub social network were taken from SNAP. The third network is a citation network from the “Physics” section on Arxiv taken from the Chen et al. research that proposes the Degree Discount heuristics [9], allowing a comparison of results between Chen et al and the this project to ensure the projects implementations are correct.

The networks were formatted to allow parsing into NetworkX objects which allows crucial functionality to the project such as finding the degree of a node. However, the process of creating a NetworkX object each time the network needs to be used was inefficient. To solve this the Python library Pickle was utilised to allow the object itself to be saved, making retrieving this network much quicker. Once the network was retrieved, it was converted to a undirected graph using the predefined NetworkX function. Since the Weighted Cascade model requires a directed graph whereas the LT and IC models need undirected graphs, ensuring a standard format in which to save them was crucial to guarantee compatibility when integrating all the components.

5.2 Models

The initial designs of the models heavily utilised the Network Diffusion Library for model implementation as it provides a very easy to use library. However, it was found early in the development stage that there was no implementation of the WC model, although this could be fixed with a slight augmentation to the IC model. Additionally, when testing of models was carried out, results and functionality were not as expected, leading to a long period of troubleshooting. The decision was then made to create new implementations of the IC and WC models within the project and only using the library for the LT model.

The IC and WC models were created using the foundations from the library’s implementation but with changes to the models that allowed the expected results to be obtained. The models contained a simple class for instantiating the necessary variables, and a single function which executed a single iteration of the model, returning the count of infected nodes.

A critical component of the instantiation of the model was the initial activation of the seed nodes, ensuring the simulation ran as intended. Additionally, defining the threshold and propagation probability in the LT and IC models respectively were important to affirm a uniform value across all diffusion within the graph.

Iterations were defined similarly for all three models, however, the following steps outline a single iteration of the IC model. For each newly activated node, each neighbour is checked for inactive status to ensure it can be activated. Then, a random sample is generated $[0,1]$ with a normal distribution and the node is activated if the random sample is less than the threshold. A copy of all node statuses is saved at the initialisation of the iteration to ensure only a single diffusion hop can occur within one iteration.

Since the model implementations only provide a method for executing a single iteration at a time, a function must be defined that allows multiple iterations. Furthermore, it must allow the model to fully diffuse until no more infections will occur to gain the most accurate result on overall influence spread. A loop is defined that will run until there have been no new infections in the current iteration through two methods, one for the IC and WC models and another for the LT model.

The IC and WC models have three states that a node can exist in: susceptible, infected, and removed. Once a node becomes infected, it has one chance with which to infect its neighbours and once it has finished it can no longer attempt to infect its neighbours again. To enforce this idea, the third state of removed is defined to prevent a infected node from making multiple infection attempts. This can be leveraged to define the exit condition of the loop, as once a full iteration has occurred where the number of infected nodes is 0, there will never be any new infections and the simulation can terminate. The number of infected nodes can then be defined as a count of nodes in the removed state.

Since LT only allows the states inactive and active, this strategy of looping till the number of infected nodes is zero does not work. Instead, the previous count of infected nodes is stored and compared against the current count. If they are equal then no new nodes have been infected during the most recent iteration, the diffusion is complete, and the count of active nodes can be used to show the number of nodes infected.

5.3 Heuristics

Development of existing heuristics was a simple process as the design and logic of each algorithm is clearly written and can be followed with ease. Although, the logic is clear translating this into code can create some challenges particularly with `arg max` which does not translate in Python and requires extra work.

The main challenges of heuristic implementation were the adaptations to take a budget instead of a seed set size. This caused many more issues than initially anticipated as it involves completely reworking how Degree Discount functions. Since the influence maximisation problem has a fixed seed set size, a loop can be defined that picks a node each iteration until the seed set is full. With budgets, this is not possible since the seed set does not have a defined number of iterations.

The solution involves using a loop with two exit conditions for checking whether the budget has been filled. Inside the loop, the nodes are sorted and iterated through, moving past each node already in the seed set until the best node not in the seed set is found. The first exit condition involves checking whether the overall cost of the current seed set is equal to the budget, in which case no further nodes can be added. The second is whether a full iteration of the inside loop has occurred without a node being added to the seed set, implying that no further nodes can be added given the remaining budget.

Random and degree costings are integer values which means they produce whole numbers that allow the budget to always be completely filled. Checking whether the overall cost of seed nodes is equal to the budget will be effective for these costing functions, but PageRank creates more challenges. Since PageRank costs based on a probability distribution, it produces costs with differing fractional values. The chance of the overall cost being equal to the budget is very low, therefore a separate check needs to be made. A copy of the seed set is made before an iteration, this is then compared to the seed set after the iteration to determine whether any nodes have been added during a entire iteration.

Novel heuristic development was an iterative process that consisted of testing numerous potential ideas and building on successful approaches to improve performance. An initial idea was to utilise a constraint on the cost of a node being added to the seed set based on a percentage of the remaining budget. This would create an exponential decrease in the cost of a node allowed in the seed set, which would attempt to remove the issue of a seed set only containing a small number of high cost nodes.

However, this did not work as effectively as intended and a different approach which involved the use of a simple degree cost ratio, defined by $\frac{c_v}{d_v}$ for a node v , to determine the

most cost effective nodes. Finding the most cost effective nodes is particularly valuable when the cost of a node is unrelated to its degree. Therefore, this heuristic produced great performance for random costing and above average for PageRank. However, since degree costing is identical to degree, if one is divided by the other the outcome is always 1. Using this heuristic with degree costing will result in a completely random seed set with all nodes being equally as cost effective.

To prevent this, degree needs to be augmented based on previous seed set additions so more and less cost effective nodes can be identified. The result of this is the PICR heuristics which produced similar performance for random and PageRank but with vastly improved performance for degree costing.

5.4 Cost Functions

When a NetworkX object is defined, each node is assigned an ID which can be used as a reference, it is utilised heavily within the heuristics for iterating through nodes. Since each node has an ID, the cost of a node can be defined efficiently as a dictionary with a node as the key and the corresponding cost as the value.

An exception to this is PageRank as NetworkX provides an implementation of the algorithm only if the graph is already in a NetworkX format. PageRank is a complex algorithm to implement so having this work already done reduced time on cost function implementation and allowed more time to be focused on heuristic development.

Additionally, creating the dictionary each time would cause computational issues as PageRank takes considerable time and random is non-deterministic so would return different costs after each execution. Pickle was utilised, similar to the networks, to allow easy switching between cost functions.

5.5 Testing

The heavy use of NetworkX objects throughout the project, allowed the passing of these objects to be much more efficient than needing to find a different way to pass a graph between functions. Additionally, the models and heuristics were defined in such a way where the seed set returned from the heuristic could be used without manipulation by the model to identify the nodes needed to be initially set as active, due to the node IDs discussed previously. However, it was discovered that the GitHub network defined the IDs as integers whereas the other two networks used strings. This caused many problems before it was identified and fixed by converting the seed set to the correct type dependent on the network currently being used.

Cost functions also used the node IDs to define its cost and therefore were simple to pass to the budgeted heuristic implementations. Since the node IDs in the cost dictionary were taken from the network itself, no issue occurred in relation to the type of the node IDs.

Testing was a combination of manual calculations and creating tests to determine if results were as expected. For example, two of the models use probabilistic approaches and therefore creating tests to prove correctness is challenging. Testing for correctness was carried out using single node seed sets and incrementally adding nodes, allowing the behaviour to be closely monitored as the size of the seed set increased. This helped uncover a number of errors including an issue where if no nodes were activated in the first iteration of the model, it would crash as an array was utilised for storing the infected node count in each iteration and an attempt to select the last element of an empty array is not possible.

Heuristic testing involved manual calculations, specifically for the Degree Discount heuristics. Prior to this, an exact replica of the experiment carried out by Chen et al. was conducted using the same network data [9]. The results could then be compared, which acted as a sanity check to ensure the heuristics worked as expected and produced results relatively similar. Initially, Degree Discount did not produce similar results which required the manual calculations to find the source of the error. The problem was a misplacement of brackets which caused the equation to be calculated in the incorrect order, an error that would have been difficult to find without manual calculations to compare against.

Integration testing of the whole system was accomplished primarily on the Arxiv citation network as it is the smallest of the three networks and therefore took the least time to perform network simulations. Additionally, the network has no properties that would create unexpected results such as it being scale-free which provides a stable environment to conduct simple testing of correctness.

Performance testing for result comparison was achieved by creating a single iteration that took a network, model and heuristic and ran the simulation with different budgets. Repetitions were then made of the diffusion simulation for the non-deterministic models and an average was taken to ensure a correct measurement of average performance. A list of results is then returned with a count of infected nodes for each size of seed set. The single iteration was run for each network, model, heuristic, and costing function combination and saved to a text file.

6 Experiments

This chapter will discuss the experiment setup used in this project. It will detail what experiments were performed to achieve reliable results and the parameters of these experiments. Parameter choice is crucial as it dictates the behaviour of the entire system and how the results from the experiments can be interpreted and compared. Each parameter will be justified fully to understand its relevance and importance within the scope of the whole project.

6.1 Parameters

seed set sizes	5-500 inclusive
threshold	0.5
propagation probability (p)	0.01
repetitions	20
budgets	50-500 inclusive * average

Table 6.1: Parameters used in experiments

Each model requires its own parameters to be defined, with Linear Threshold requiring a threshold to be specified and Independent Cascade needing a propagation probability to be stated. The threshold is the percentage of a node’s neighbours that need to be infected before the node itself becomes infected. If the threshold is set too high, influence will not spread well resulting in a low number of infected nodes but if its set too low, all nodes in the network will become infected and a comparison will not be possible. A threshold of 0.5 provides a good balance where influence is spread well without infecting too large a percentage of the total nodes in the network.

The propagation probability is used in IC to determine the probability an infected node will infect its neighbour. Previous research such as Chen et al. used 0.01 as p to ensure influence spread was sensitive to different heuristics, allowing a comparison to be made [9]. Although, 0.02 or 0.05 could also be used, to keep consistent with research in the field, the probability of 0.01 was selected. This parameter was additionally important as it was utilised in both Degree Discount and PICR heuristics within the discounting formula.

Additionally to model specific parameters, a framework was also required when detailing how the experiment itself was structured. A number of seed sets and a budgets need to be defined for our uniform and non-uniform tests respectively. Additionally, the experiment itself must be strictly defined to ensure exactly the same environment for each heuristic.

The original uniform cost tests used seed set sizes of 50 to 500 in increments of 50. Previous research such as Chen et al [9, 8] used a range of 1 to 50 incrementing by 1 each time but this produced relatively low influence spread due to the large scale of the social networks chosen for this project. A larger seed set also allowed for more opportunities for different heuristics to choose different nodes to add as seeds, creating more diversity between the seed set each heuristic produced.

When considering budgets, similarity needs to be kept with the uniform costing, meaning finding a way of translating seed set sizes of 50 to 500 to budgets. The average was already utilised in the average cost heuristic to define the median cost in each network. The average was employed in conjunction with the seed set sizes to define a budget that will, on average, add the same number of nodes as the uniform cost tests. If the average calculated from each network-cost function pairing, found in 6.2, is multiplied by each

seed set size 50 to 500, the result is a budget that adds a similar number of nodes to the seed set as its uniform cost counterpart. The averages were calculated for each individual network as there were deviation between them.

	Random	Degree	PageRank
Amazon	5	6	0.0000033
GitHub	5	6	0.000013
Arxiv	5	4	0.000027

Table 6.2: Median cost of each network for each costing function

6.2 Experiment Setup

To ensure integrity of results, it is imperative that minimal changes were made throughout the testing phases. Uniform cost testing was carried out first as it required less implementation due to the seed set sizes being fixed across all tests. By defining networks, diffusion models, and heuristics in arrays, each can be iterated through within one function to test every potential combination. Running these tests in one execution had the benefit of preventing a combination from being missed, making the results incomplete. However, it had the drawback of taking considerable time to run and was unable to be interrupted mid-execution without the need for a complete restart.

Additionally, within each test of a combination of network, model, and heuristic, 10 different seed set sizes were required to be iterated through to achieve thorough results. The importance of this was within each seed set size, the model simulation needed to be repeated a number of times and then an average of the results taken to obtain accurate performance from probabilistic models. Initial testing was implemented using 50 repetitions for each seed set size, however, this was found to cause single combinations to take days to complete which was impractical with the time constraints imposed on the project. Additionally, more repetitions did not necessary change the result from calculating the average after around 20 repetitions. Therefore, the repetition parameter was changed to 20 and although run-time was still high, it caused a reduction to a manageable level.

Budgeted tested requires a costing function to be incorporated into the experiment design. Initially, the network was static and each costing function was iterated through until all tests on a specific network had completed. However, this further increased the run-time of a single execution since each cost function needs to run every pair of model and heuristic. Hence, a new solution was developed where the a full test iteration was run with the same cost function throughout the entire execution. All tests for a single cost function were run to completion before the cost function was manually updated and the tests executed again. Although this required changing the cost function between test execution, a minimal change was judged as unlikely to affect the quality of results produced by these tests.

A summary of the key parameters used in the project can be seen in Table 6.1. The overall aim of these parameters was to obtain results that could be effectively used for comparison and evaluation.

7 Results

This chapter will discuss the results obtained from the experiment described in Chapter 6. Uniform and Non-uniform experiments were conducted but with the project focus on the budgeted influence maximisation problem, there will be less focus on the results of the uniform experiments. Notable data comparisons will be made in addition to evaluating of reasoning behind results obtained with the full results shown in Appendix A.

7.1 Max Influence Spread

As expected, the baseline heuristics, random and degree, do not perform well in the BIM problem. Additionally, the IM designed heuristics Single Discount and Degree Discount behave identical if not worse than the baseline heuristics particularly in costing functions that are correlated with node degree. However, these heuristics do have good performance within a specific used case of the random costing function. The heuristics degree, Single Degree, and Degree Discount achieve influence spread to every node with the GitHub network and Linear Threshold model. This is the only occurrence of a heuristic accomplishing maximum diffusion across all experiments.

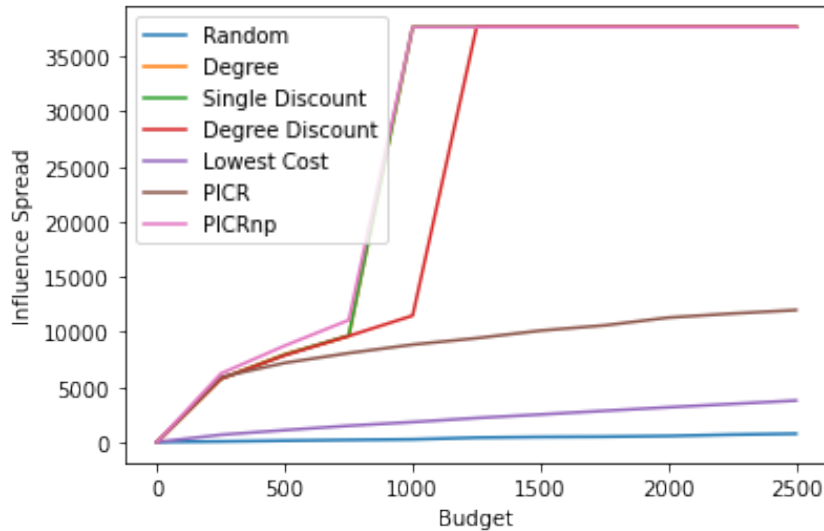


Figure 7.1: Results for the GitHub Network using random costing on the Linear Threshold model.

An explanation of this unique occurrence can potentially be gained from the general network properties of the GitHub graph. As seen in Figure 7.2, the degree distribution has been fitted to a power-law distribution using the power-law Python library [3]. The plot shows that the distribution of the GitHub network appears to be strongly scale-free as it follows the power-law relationship very closely. Scale-free networks are networks with a low number of very high degree nodes and a extremely large number of low degree nodes. Scale-free networks are common with online social networks and given GitHub fits this criteria, its scale-free property is not unusual [7].

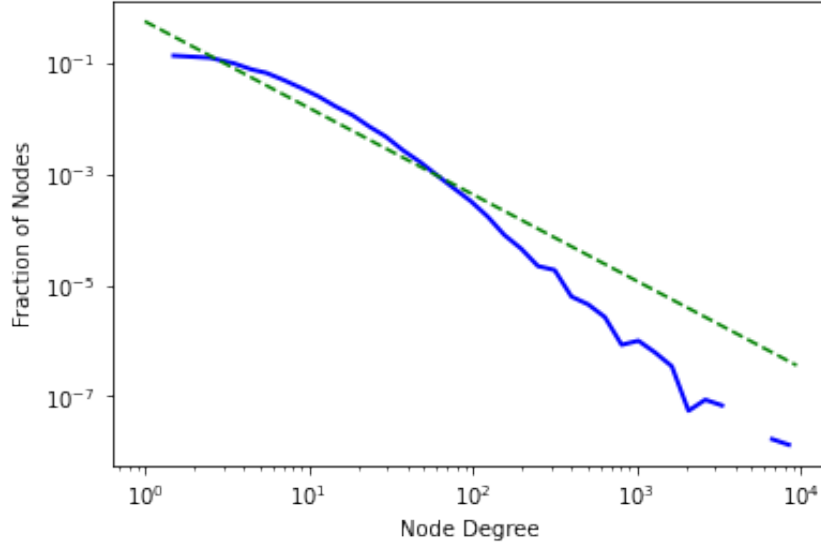


Figure 7.2: Graph showing power-law with node degrees on the x-axis and the fraction of nodes at each degree on the y-axis.

The green line displays a power law relationship and the blue line shows the degree distribution of the GitHub network.

Influence spread on this scale only becomes possible when the costing of a node has no relation to its degree or potential influence. This allows heuristics such as degree to add many nodes to the seed set without these nodes having large costs. The combination of the random costing plus the scale-free property of the network creates the perfect environment for degree focused heuristics to cause maximum influence dissemination. However, the PICRnp heuristic also produces these results which contradicts this explanation as it is not purely based on degree and utilises cost within its potential spread calculation.

A complete influence of every node in the network is achieved in the fourth budget increase iteration. With a budget of 1000, the degree heuristic is able to add 199 nodes to the seed set, meaning any additional nodes added as seeds are unnecessary as influence cannot be increase. The 199 highest degree nodes in the Github network are therefore enough to cause influence to spread to every node in the network which is only possible due to its scale-free nature. Additionally, PICRnp with the same budget adds 394 node to the seed set. Since the heuristic has a cost element, its behaviour is different to the purely degree-based heuristic. Instead of adding the highest degree nodes while ignoring their cost, the PICRnp heuristic takes cost into consideration, only adding a node at the next highest cost if its expected influence is double that of the best node at the current cost.

An example of this is with three nodes, node A has a potential influence of 10 and a cost of 1, node B has potential influence 15 with cost 2, and node C has potential influence 40 with cost of 3. Since node C has the best potential influence to cost ratio even though its cost is also the highest, it will be added to the seed set first. Second, node A will be added because although it has less potential influence than node B, it is more cost effective. Node B would need to have a potential influence over double node A to be added to the seed set since its cost is 1 greater. Within the larger PICRnp seed set, the seed set from the degree heuristic is a direct subset of the PICRnp seed set. Even with the different heuristic approaches, due to the large budget and the inclination of the PICRnp heuristic to add smaller cost nodes allowing more nodes to fit in the budget. Both heuristics come to the same solution for causing maximum influence spread.

There is little deviation between the four heuristics that achieve the same influence spread, however, Degree Discount takes a fifth budget increase to reach maximum diffusion. This is likely due to the heuristic discounting high degree nodes that are pivotal in spreading influence, meaning more iterations are required before they are added as seeds.

Although, PICRnp reaches maximum influence, the twin heuristic utilising probability only achieves 30% of the influence. With probability, the heuristic becomes less degree focused and more about probabilistic influence spread. Since degree is the key property causing the greatest influence, the more degree focused PICRnp is more suited to the task. Additionally, Linear Threshold is not a probabilistic model and although another heuristic utilising propagation probability, degree discount, performed well in testing carried out by Chen et al. [9] using a probabilistic heuristic with a non-probabilistic model is likely to not produce optimal results.

7.2 Lowest Cost Performance

Over all 27 budgeted test combinations, the Lowest Cost heuristic had the best performance in just under 50% of cases, including all tests involving Weighted Cascade. It is the heuristic with the single highest best performance percentage over all the test cases, showing that in general it is an effective strategy to pick a very large number of low costing nodes to add to the seed set. A marketing approach of this nature would be described as “directed marketing” where individuals are marketed to, regardless of the influence they might spread. What this result shows is that in a large proportion of cases, the best approach to maximise influence spread does not involve trying to utilise the concept of influence spread and instead focuses on adding as many nodes as possible as seeds.

Additionally, the Lowest Cost heuristic provides the best results for every test where the model used is WC except for one. Weighted Cascade was defined by Kempe et al to solve the problem in Independent Cascade of bias towards higher degree nodes due to having a fixed propagation probability [19]. This means lower degree nodes have higher propagation probabilities and, with degree and PageRank costings, have lower costs. Combining a higher propagation probability with a lower cost, allows Lowest Cost to pick a large number of low degree and low cost nodes and have them spread potentially more influence than larger degree nodes.

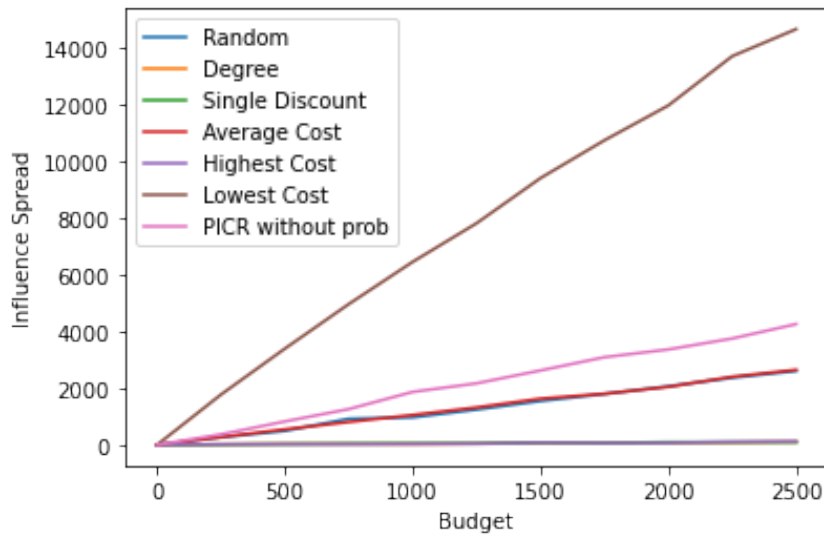


Figure 7.3: Results for the Amazon Network using degree costing on the Weighted Cascade model

The Weighted Cascade environment where Lowest Cost does not obtain the best results is the Arxiv network with degree costing, being beaten by the PICRnp heuristic. Both heuristics only add one cost nodes to the seed set but the approach taken by the PICR heuristic employs a slight improvement. A scenario could occur where multiple single degree nodes are connected together, as shown in Figure 7.4.

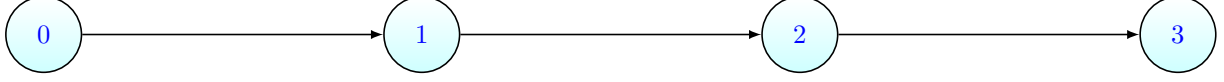


Figure 7.4: Chain of directed nodes

In Lowest Cost, if node 1 is added to the seed set, then there is still the possibility of node 0 also being added in future heuristic iterations. Since node 1 is already in the seed set and therefore infected, node 0 cannot cause any additional influence. However, PICR discounts neighbours already in the seed set, therefore if node 1 was added as a seed, node 0 would never be added since its degree will be discounted from 1 to 0. This small adaption over a large budget can create noticeable differences in overall seed nodes, leading to a visible difference in overall influence spread.

The behaviour of PICR both with and without probability is not unique to this single use case as this is observed across all testing involving the degree cost function. Between the two heuristics, the performance achieved is almost identical if not better than that of the Lowest Cost heuristic on all test cases with degree costing. This is most clearly seen in with the GitHub network and Linear Threshold model, again using degree costing. Since Linear Threshold is non-deterministic, direct comparisons can be made without the issue of randomness. Lowest Cost, PICR, and PICRnp all return the same result for every budget size, showing that they all add exactly the same nodes. It is important that they add exactly the same nodes as having identical seed sets is not found in any other test case. Additionally, the results from IC and WC for the same network and cost function are within a margin of error to each other. A conclusion can therefore be drawn that the cause of this is again the GitHub network. The scale-free property of the network was previously discussed as it caused maximum influence spread under the random costing function. Adding to this, having large hub nodes connected to many low or single degree nodes removes large chains of single degree nodes connected together. Therefore, the benefit from discounting, found in PICR, is negligible within this environment. This can be proven further by examining results from these three heuristics using the Amazon and Arxiv networks. The differences found in these results are vast, meaning the cause can only be the one changed variable which is the network.

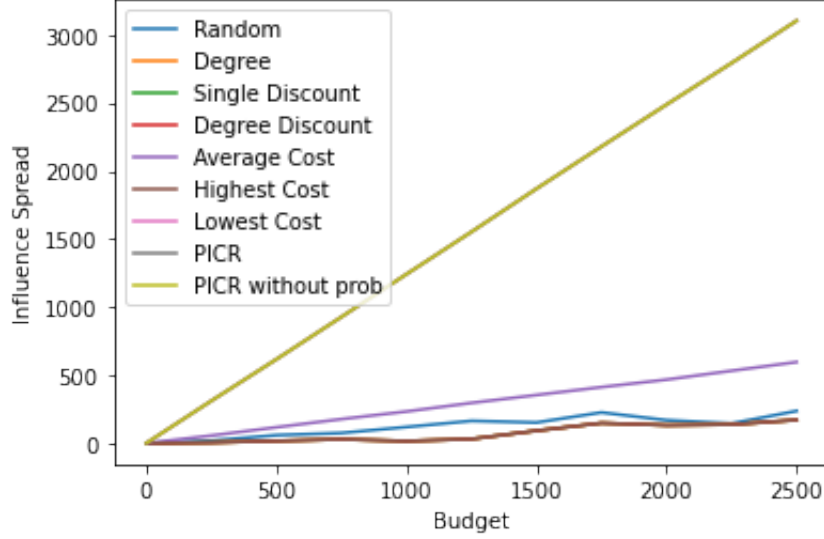


Figure 7.5: Results for the GitHub Network using degree costing on the Linear Threshold model

7.3 Highest and Average Cost Performance

There are no test cases where Highest Cost or Average Cost provide the best influence maximisation. As expected, the performance of the heuristics with random costing is very poor, obtaining results similar to random node selection. Highest Cost performs badly across all cost functions even when cost is correlated to potential influence. Degree and PageRank findings are very similar to simple degree or Single Degree heuristics, showing there is little benefit in a marketing strategy of adding only the most expensive nodes.

Average Cost, although has issues with the random cost function, returns better results with degree and PageRank costing. Results are better than previously implemented heuristics such as Degree Discount, however, are significantly worse than Lowest Cost results. Since, it has been established that adding many low cost, low degree nodes is a more effective approach than adding higher degree nodes. The approach of Average Cost is directly inferior to Lowest Cost as adding nodes with higher costs creates a seed set with less nodes, leading to lower overall influence spread.

7.4 PICR and PICR without probability

The PICR heuristics offer different behaviour dependent on the cost function used. The behaviour with degree costing of adding only 1 cost nodes has already been discussed, however, there are still two further cost functions.

Random costing has also been briefly examined but only in the context of a single test case example. The heuristics will define a potential influence to cost ratio for each node, for random costing this will generally result in a similar outcome to filtering all 1 cost nodes and sorting by degree. Higher costing nodes such as 2 cost nodes will be ranked with 1 cost nodes with half their degree and 3 cost nodes with one third of their degree, this occurs for each cost bracket. Since the most cost effective nodes relative to degree are being added, the results produced are consistently good for all random cost tests. However, these get outperformed frequently by the Lowest Cost heuristic, again showing the value of adding only lowest cost nodes over a more complex approach.

Since PageRank costing is similar to degree, the methodology of the heuristics is similar but with the difference that PageRank is a probability distribution and therefore the costs and budgets are not whole numbers. Since cost and degree are not as closely

related in PageRank, nodes with degrees greater than 1 are added by both Lowest Cost and PICR heuristics. However, a comparison on the seed sets produced with the Arxiv network show vast differences between PICR, PICRnp, and Lowest Cost.

PICR and Lowest Cost have similar sized seed sets with 1541 and 1542 respectively, however, a comparison of the seed sets shows a difference of 30 nodes. Observations on the seed sets show the beginning of the sets are identical but deviations start to occur as more nodes are added. The reason for this variation can be only from the discounts made to the degree of nodes in PICR and PICRnp heuristics.

Knowing this, a very interesting observation can be made from the results. Lowest Cost and PICR have the same influence at the maximum budget for LT and are within a margin of error for IC, even though their seed sets are different. Additionally, their influence for each budget incrementation differs for around 50% of budgets, further proving that the seed sets produced are indeed different. Identical influence spread can also be seen in the Amazon and GitHub networks with PageRank costing. The cause of this is unclear, however, the extra steps of the PICR heuristic to discount degrees appears to have no effect on overall influence spread, even if they cause different nodes to be added as seeds. Results from PageRank costing of Lowest Cost, PICR, and PICRnp are seen in Table 7.1.

Network	Model	Lowest Cost	PICR	PICRnp
Amazon	Linear Threshold	1209	1209	1546
	Independent Cascade	1229	1230	1622
GitHub	Linear Threshold	1243	1243	68
	Independent Cascade	3489	3542	2313
Arxiv	Linear Threshold	1546	1546	395
	Independent Cascade	1622	1638	798

Table 7.1: Results for PageRank costing function with Linear Threshold and Independent Cascade models

A further observation made is PICRnp gives a seed set of only 395 nodes with the only difference to PICR being the removal of the probability multiplier. Moreover, it produces vastly different result, for the GitHub network under the LT model the results are one tenth of the PICR results whereas the same model with the Amazon network produces the best results out of any heuristic. When probability is removed, the heuristic becomes purely the ratio between the degree and the cost. Since PageRank is not completely correlation to the degree, there is an advantage to only looking at the degree to cost ratio as PageRank might undervalue nodes that have higher degrees. In the same way degree will cost all nodes only considering a single network property, regardless of properties such as centrality. This makes PICRnp better than other heuristics in this environment as the lowest cost nodes in PageRank are more likely to be the very worst nodes in the network.

Furthermore, the results for degree costing show that Lowest Cost and PICR have vastly better performance to PICRnp in the Amazon network with LT model. Therefore, the reasoning for the PICRnp performance with the PageRank costing is solely down to the cost function.

7.5 Differences Between Degree and PageRank

The differences between how degree and PageRank handle node costing gives great insight into the shortfall of degree costing and the benefits of PageRank. Whereas degree costs

nodes using a single network property, PageRank considers more than just the number of neighbours a node has. Since degree only considers a node's neighbours, any properties outside of this small subgraph are not considered. A single degree node could be located very centrally within the network, therefore making it much more effective at spreading influence than a single degree node at the edge of the graph, however, both would have the same cost associated with them. In figure 7.6 nodes A and C will have a cost of 1 under the degree costing but once node A infects node B that is the end of the diffusion node A can cause. Whereas if node C infects node D there are many possible scenarios where the diffusion is continued throughout the network.

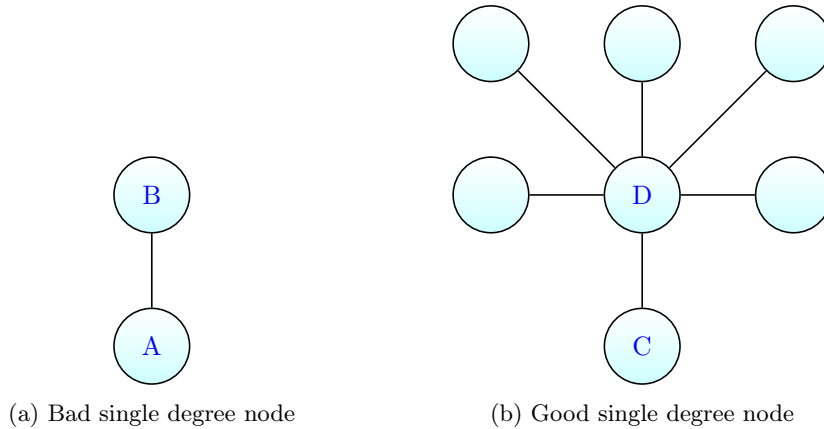


Figure 7.6: Different scenarios for single degree nodes

In this situation, PageRank will cost the node A much less than node C since it is not just observing neighbours and instead using its iterative process of moving around value based on connections. This means that the value that will be given to node C from node D will be much larger than node B to node A because node D will have accumulated more value from having more neighbours.

Through this example, there are scenarios where degree will undervalue nodes due to its naivety. If this can be exploited through a heuristic then good performance could be obtained within the degree costing space. However, a heuristic of this nature will not generalise well to other methods of costing such as random or PageRank. Conversely, PageRank has the potential to undervalue nodes with relatively high degrees as it considers wider importance over purely neighbour count. Although the benefits of this are not as sizable as degree costing, a higher degree node that is not well connected can still diffuse influence locally to its neighbours. This could lead to better influence spread than simply adding only one degree nodes at the same cost.

The difference in costing functions can be seen clearly in the results produced by the heuristics degree and Single Discount as they are purely utilising degree for node selection. These heuristics perform better on PageRank costing with the Amazon and Arxiv networks and have similar performance for the GitHub network. As seen in figure A.3 and figure 7.3, the heuristics perform considerably better with two of the networks. Since the GitHub network is scale-free, PageRank will overvalue the very high degree nodes compared to degree costing as they are hub nodes within the network and therefore will be assigned a very high value due to majority of random click scenarios terminating on one of these hub nodes.

Network	Model	Degree	PageRank
Amazon	Linear Threshold	133	168
	Independent Cascade	63	87
	Weighted Cascade	107	162
GitHub	Linear Threshold	171	158
	Independent Cascade	2267	2173
	Weighted Cascade	2282	2235
Arxiv	Linear Threshold	60	231
	Independent Cascade	159	406
	Weighted Cascade	411	808

Table 7.2: Degree heuristic results

Network	Model	Degree	PageRank
Amazon	Linear Threshold	133	168
	Independent Cascade	64	90
	Weighted Cascade	94	159
GitHub	Linear Threshold	171	158
	Independent Cascade	2214	2206
	Weighted Cascade	2272	2304
Arxiv	Linear Threshold	59	235
	Independent Cascade	162	430
	Weighted Cascade	437	909

Table 7.3: Single Degree heuristic results

8 Conclusion

This section presents the research findings obtained from different heuristic approaches to solve the budgeted influence maximisation problem. The aim of the project was to identify the effectiveness of previous uniform cost heuristics in the budgeted environment and the performance of new heuristics designed specifically for the budgeted influence maximisation problem.

The overall influence spread caused by each heuristic’s seed set was measured using three real-world social networks, three diffusion models, and three costing functions. Every combination of network, model, and costing function provided distinctly different behaviour to provide comprehensive results. The seed sets produced by 9 heuristics were tested for each combination using incrementally increasing budget sizes.

8.1 Findings

Data was collected and analysed using a number of data analysis techniques to discover trends and discrepancies in findings. Overall results show that for all test cases either Lowest Cost or PICR heuristics provided the best performance in terms of influence maximisation. However, due to unique network properties and model behaviour, different combinations identified different heuristics as displaying the best performance.

Lowest Cost is best paired with the Weighted Cascade model with any network and costing function as the WC model benefits heavily from adding a large number of nodes as seeds due to its greater effectiveness of spreading influence through low degree nodes. Since WC bases propagation probability on node degree, having a low degree increases overall propagation probability.

The scale-free property of the GitHub network was also examined to understand the effects it had on the heuristic performance. Most notably, maximum influence was reached on the LT model when the costing was random on all purely degree focused heuristics. This is important as viral marketing is usually conducted on online social networks which have a higher propensity to be scale-free [7]. Using a purely degree approach in this scenario can therefore lead to greater idea spread and subsequently, potentially greater sales.

PageRank’s effectiveness in costing nodes compared to degree uncovers the disadvantages associated with undervaluing nodes. With PageRank considering a nodes place in relation to the entire network allows for more effective costing than degree’s approach of only considering a node’s neighbours. Under the constraint of only degree costing, this can be exploited by selecting single degree nodes with high degree neighbours, allowing for high influence spread without paying a premium for the high degree node itself. However, with the addition of PageRank this loophole is closed, and a more generalised approach must be taken.

In real-world marketing scenarios where the cost of paying an individual to market a product is based on, for example, the number of followers they have. The approach of marketing to a low follower neighbour would not be effective as more factors are involving in an individuals pricing that simply followers. However, there is the possibility for this to be adapted to simulate more closely this scenario.

8.2 Future Work

Time constraints caused many limitations on implementation of networks, diffusion models, and heuristics. Since complex social networks can vary greatly in underlying properties and architecture the use of only three does not encapsulate heuristic performance

with other real-world examples. Due to the viral marketing scenarios discussed, real-world examples of networks used in viral marketing campaigns could provide great insight into how heuristics compare in an actual example.

Many diffusion models have been proposed including variations on the three implemented in this project. Extensions to the Linear Threshold model such as Kertesz Threshold could be implemented to explore the heuristics under a threshold model with slightly different behaviour. Additionally, a plethora of improvements on the Independent Cascade model exist exploring different scenarios such as time-awareness.

The project attempted to implement a suitable range of heuristics with which to test the newly developed approaches. However, work on heuristics is vast and only a sample could be included. Considerable work could be done to improve the heuristic selection, with more complex uniform cost heuristics such as PMIA and LDAG and even non-heuristic greedy approaches.

Additionally, heuristic work has been performed in solving budgeted influence maximisation such as Billboard, Handbill and comminatory heuristics proposed by Han et al. [16] in addition to Nguyen and Zheng proposed heuristics. The implementation of these would add considerable depth to the comparison as it would allow a direct comparison against heuristics developed specifically with the budgeted influence maximisation problem in mind.

9 Evaluation

This chapter will discuss the success and effectiveness of the project with respect to the overall aim of providing solutions to the budgeted influence maximisation problem. In addition to how well the project achieved its goals, the development is also evaluated due to its major contribution to the overall project.

9.1 Project Evaluation

The main aim of the project was to implement a testing suite that would allow for the performance of different heuristics to be measured and compared against each other. It was further specified that networks, models, heuristics, and cost functions needed to be developed to achieve this.

A test suite was successfully developed through the implementation of the four components discussed in Chapter 3. Within these four sections, changes were made to accommodate the time constraint associated with the project and the computation complexity of running simulations. Initially, heuristics such as PMIA and LDAG were planned to be implemented but due to their inability to work universally across all diffusion models in addition to time constraints, development was focused on more crucial components.

Additional issues were encountered during the development particularly during integration of components that caused extra, unexpected, work. This is a problem typical with implementation of this scale and was managed effectively by the risk mitigations put in place at the beginning of the project. Additional issues were found when passing seed sets to models due to the integer nature of node IDs preventing the correct nodes from being initially infected. This caused major issues for one of the three networks utilised but was remedied through continuous testing and debugging.

For some experiments of the non-deterministic IC and WC models, simulations needed to be run numerous time to obtain effective results. This caused high run-times with some experiments on the larger Amazon network taking days to execute. Unfortunately, this could not be fully avoided but could be negated slightly through parallelisation, but this was only considered as an option after testing had completed. The solution taken was to reduce the number of simulations from 50 to 20 as the difference in results between the two was found to be negligible. This reduced the run-time by over half, making it feasible for the limited number of tests being conducted.

Storing of results would have benefited from more preplanning, organisation, and automation to ensure all needed tests were conducted. Results throughout this project were written to files by the tests after each combination of network, model, heuristic, and cost function was completed. This was an adequate organisation method for a small handful of initial results but the results from tests grew to be very large due to the addition of more heuristics. If a database had been established before any testing was run, a simple interface between the database and Python could have been created, allowing results to be easily stored and accessed when analysis was needed. Additionally, if testing had been better automated, it could be conducted in batches, allowing the correct parameters to be selected each time and removing the risk of human error.

9.2 Research Evaluation

In addition to the development of a test suite for simulating influence spread through networks for a number of different heuristics. New Heuristic approaches were developed in a novel way that relied solely on cost to select seed nodes. Additional methods were also implemented that utilised cost as well as the potential influence spread using propagation

probabilities. Although research on heuristics for the budgeted influence maximisation problem is relatively thorough, there is little to no research on the sole utilisation of cost for seed set selection. The concept of using a ratio of potential influence to cost is also novel and has not been discussed in previous literature.

The project achieved its research goal by allowing valuable heuristic comparison with regard to influence maximisation and showed that the novel approaches shown in this paper are indeed more effective than previously defined uniform cost heuristics. And, although some results were as expected such as the poor performance of Highest Cost and random heuristics, interesting results such as the effectiveness of the Lowest Cost heuristic were also identified.

There were minimal implementation issues, and all were able to be solved using mitigation set out at the beginning of the project. The only significant challenge was the diffusion model implementations due to the library not working as expected but this was overcome.

The project was completed within the time frame and without the need to revise the overall aims and objectives of the project. It also sets out potential future research in the area of more cost function focused heuristics for the budgeted influence maximisation problem.

References

- [1] E. T. Aaron Clauset and M. Sainz. The colorado index of complex networks. 2016.
- [2] P. Adrian and K. Nicole. An experiential, social network-based approach to direct marketing. *Direct Marketing: An International Journal*, 3(3):162–176, Jan 2009.
- [3] J. Alstott, E. Bullmore, and D. Plenz. powerlaw: A python package for analysis of heavy-tailed distributions. *PLOS ONE*, 9(1):1–11, 01 2014.
- [4] D. Arthur, R. Motwani, A. Sharma, and Y. Xu. Pricing strategies for viral marketing on social networks. In S. Leonardi, editor, *Internet and Network Economics*, pages 101–112, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.
- [5] J. A. Barnes. Class and committees in a norwegian island parish. *Human Relations*, 7(1):39–58, 1954.
- [6] S. Brin and L. Page. The anatomy of a large-scale hypertextual web search engine. *Computer Networks and ISDN Systems*, 30(1):107–117, 1998. Proceedings of the Seventh International World Wide Web Conference.
- [7] A. D. Broido and A. Clauset. Scale-free networks are rare. *Nature Communications*, 10(1), Mar 2019.
- [8] W. Chen, C. Wang, and Y. Wang. Scalable influence maximization for prevalent viral marketing in large-scale social networks. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD ’10, page 1029–1038, New York, NY, USA, 2010. Association for Computing Machinery.
- [9] W. Chen, Y. Wang, and S. Yang. Efficient influence maximization in social networks. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD ’09, page 199–208, New York, NY, USA, 2009. Association for Computing Machinery.
- [10] P. Domingos and M. Richardson. Mining the network value of customers. In *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD ’01, page 57–66, New York, NY, USA, 2001. Association for Computing Machinery.
- [11] A. Goyal, W. Lu, and L. Lakshmanan. Celf++: Optimizing the greedy algorithm for influence maximization in social networks. volume 47-48, pages 47–48, 01 2011.
- [12] M. Granovetter. Threshold models of collective behavior. *American Journal of Sociology*, 83(6):1420–1443, 1978.
- [13] E. Güney. On the optimal solution of budgeted influence maximization problem in social networks. *Operational Research*, 19(3):817–831, 2019.
- [14] A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using networkx. In G. Varoquaux, T. Vaught, and J. Millman, editors, *Proceedings of the 7th Python in Science Conference*, pages 11 – 15, Pasadena, CA USA, 2008.
- [15] O. Hall. Medical Innovation: A Diffusion Study. By James S. Coleman, Elihu Katz, and Herbert Menzel. Foreword by Joseph A. Precker. Indianapolis: The Bobbs-Merrill Company, 1966. 246 pp. Illustrated. Paper. *Social Forces*, 46(2):291–291, 12 1967.

-
- [16] S. Han, F. Zhuang, Q. He, and Z. Shi. Balanced seed selection for budgeted influence maximization in social networks. volume 8443, pages 65–77, 05 2014.
- [17] M. O. Jackson. The economics of social networks. 2005.
- [18] A. E. Kazak and R. S. Marvin. Differences, difficulties and adaptation: Stress and social networks in families with a handicapped child. *Family Relations*, 33(1):67–77, 1984.
- [19] D. Kempe, J. Kleinberg, and E. Tardos. Maximizing the spread of influence through a social network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '03, page 137–146, New York, NY, USA, 2003. Association for Computing Machinery.
- [20] S. Khuller, A. Moss, and J. S. Naor. The budgeted maximum coverage problem. *Information Processing Letters*, 70(1):39–45, 1999.
- [21] A. Krause and C. Guestrin. A note on the budgeted maximization of submodular functions. 01 2005.
- [22] J. Leskovec, L. A. Adamic, and B. A. Huberman. The dynamics of viral marketing. *ACM Trans. Web*, 1(1):5–es, May 2007.
- [23] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. VanBriesen, and N. Glance. Cost-effective outbreak detection in networks. In *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '07, page 420–429, New York, NY, USA, 2007. Association for Computing Machinery.
- [24] J. Leskovec and R. Sosič. Snap: A general-purpose network analysis and graph-mining library. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 8(1):1, 2016.
- [25] J. C. Mitchell. Social networks. *Annual Review of Anthropology*, 3(1):279–299, 1974.
- [26] H. Nguyen and R. Zheng. On budgeted influence maximization in social networks, 2013.
- [27] M. Richardson and P. Domingos. Mining knowledge-sharing sites for viral marketing. In *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '02, page 61–70, New York, NY, USA, 2002. Association for Computing Machinery.
- [28] G. Rossetti, L. Milli, S. Rinzivillo, A. Sirbu, F. Giannotti, and D. Pedreschi. Ndlb: a python library to model and analyze diffusion processes over complex networks. *International Journal of Data Science and Analytics*, 2018, Dec. 2017.
- [29] G. Rossetti, L. Milli, S. Rinzivillo, A. Sirbu, D. Pedreschi, and F. Giannotti. Ndlb: Studying network diffusion dynamics. pages 155–164, Tokyo, Japan, 2017. IEEE.
- [30] R. Wang, W. Zhang, H. Deng, N. Wang, Q. Miao, and X. Zhao. Discover community leader in social network with pagerank. In Y. Tan, Y. Shi, and H. Mo, editors, *Advances in Swarm Intelligence*, pages 154–162, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.
- [31] S. Wasserman and K. Faust. *Social Network Analysis: Methods and Applications*. Structural Analysis in the Social Sciences. Cambridge University Press, 1994.

A Full Results

Random Costing									
	Amazon			GitHub			Arxiv		
	LT	IC	WC	LT	IC	WC	LT	IC	WC
Random	524	560	2815	755	2773	3586	698	654	2354
Degree	1915	1164	3347	37657	3061	3004	828	967	1539
Single Degree	1864	1156	3401	37657	3107	3007	919	1133	1941
Degree Discount	1755	1160	N/A	37657	3154	N/A	923	1201	N/A
Average Cost	543	556	2826	600	2732	3631	765	656	2120
Highest Cost	307	351	1534	1110	2620	3057	539	333	1118
Lowest Cost	3011	2936	10424	3769	5121	7083	4114	3096	7664
PICR	3032	3307	N/A	11978	4899	N/A	3582	3307	N/A
PICRnp	4114	3096	7664	37657	3584	3801	2065	2325	4206

Table A.1: Degree heuristic results

Degree Costing									
	Amazon			GitHub			Arxiv		
	LT	IC	WC	LT	IC	WC	LT	IC	WC
Random	446	484	2606	237	2393	3051	432	402	1514
Degree	133	63	107	171	2267	2282	60	159	411
Single Degree	133	64	94	171	2214	2272	59	162	437
Degree Discount	133	65	N/A	171	2234	N/A	61	159	N/A
Average Cost	561	556	2644	596	2736	3684	781	525	1628
Highest Cost	133	66	122	171	2192	2277	55	161	438
Lowest Cost	3188	3049	14651	3103	5268	8681	2179	2014	4256
PICR	3188	3050	N/A	3103	5227	N/A	3137	2034	N/A
PICRnp	2179	2014	4256	3103	5242	8630	3137	2035	5804

Table A.2: Degree heuristic results

PageRank Costing									
	Amazon			GitHub			Arxiv		
	LT	IC	WC	LT	IC	WC	LT	IC	WC
Random	443	485	2630	372	2541	3007	588	518	2070
Degree	168	87	162	158	2173	2235	231	406	808
Single Degree	168	90	159	158	2206	2304	235	430	909
Degree Discount	168	89	N/A	158	2201	N/A	224	440	N/A
Average Cost	538	531	2675	572	2774	3720	508	501	505
Highest Cost	188	90	160	158	2240	2257	460	250	717
Lowest Cost	1209	1229	7298	1243	3489	4696	1546	1622	5512
PICR	1209	1230	N/A	1243	3542	N/A	1546	1638	N/A
PICRnp	1546	1622	5512	68	2313	2326	395	798	1239

Table A.3: Degree heuristic results